COLLEGE SAHAYAK

Government Polytechnic Awasari (Khurd)

Department of Computer Engineering

# Operating System

Course Code: 315319

## Complete Unit-Wise Class Notes

Semester 5 | K-Scheme | CO5I

Academic Year 2025–26

Units I • II • III • IV • V + PYQ Examples

| Unit | Topic | Marks |
|---|---|---|
| I | Operating System: Services & Components | 14 |
| II | Process Management | 14 |
| III | CPU Scheduling | 14 |
| IV | Memory Management | 14 |
| V | File Management & Disk Scheduling | 14 |

# UNIT I Operating System: Services & Components

*CO1: Explain the services and components of an Operating System. Total Marks: 14*

**Syllabus: Total Marks:14**

## Operating System: concept, functions:

- An Operating System (OS) is a system software that acts as an intermediary between a computer's hardware and the user.
- It enables users and application programs to interact with the hardware in a consistent and controlled way.

### Concept of an Operating System:

- An Operating System is system software that manages computer hardware, software resources, and provides common services for computer programs.
- It is the first software loaded into memory when a computer is started (booted) and it remains in operation as long as the system is on.

### Functions of an Operating System:

Here are the key functions of an OS:

- Process Management
- Manages processes in a system (creating, scheduling, and terminating processes).
- Handles multitasking (running multiple programs at once).
- Provides mechanisms for process synchronization and communication.
- Memory Management
- Keeps track of each byte in a computer’s memory.
- Allocates memory to processes and reclaims it when no longer needed.
- Manages virtual memory to allow more efficient and larger memory use.
- File System Management
- Controls the creation, deletion, and access of files.
- Organizes data into files and directories.
- Maintains file permissions and security.
- Device Management
- Manages device communication via drivers.
- Facilitates input/output operations between hardware and software.
- Keeps track of all peripherals (e.g., printers, hard drives).
- User Interface
- Provides interfaces like Command Line Interface (CLI) and Graphical User Interface (GUI) for interaction.
- Allows users to control and interact with the system easily.
- Security and Access Control
- Protects data and system resources from unauthorized access.
- Implements user authentication and access permissions.
- Resource Allocation
- Allocates hardware resources (CPU, memory, I/O devices) to various programs and users.
- Ensures efficient and fair use of system resources.
- Networking
- Supports communication between devices and systems over networks.
- Manages protocols and data exchange.

College Sahayak

■ Examples of Popular Operating Systems

**Windows (Microsoft)**

**macOS (Apple)**

**Linux (Open-source, various distributions)**

**Android (based on Linux, for mobile devices)**

**iOS (Apple's mobile OS)**

---

## Different types of Operating System:

There are several types of Operating Systems (OS), each designed for specific types of computing environments and tasks. Following are the types of operating system:

- Batch Operating System
- Multi-programmed
- Time Shared Operating System
- Multiprocessor Operating System
- Distributed Operating System
- Real Time Operating System
- Mobile OS (Android OS).

### Batch Operating System

- A Batch Operating System is one of the earliest types of operating systems, primarily used in mainframe computers during the 1950s and 60s.
- It was designed to process jobs in batches with minimal or no interaction from the user during execution.
- A Batch Operating System collects programs and data together in a batch before processing starts.
- These jobs are submitted to the computer operator, who groups them and feeds them into the computer as a single batch.
- The OS then processes each job sequentially, without user intervention.
- Batch systems are still used in:
- Payroll processing
- Banking systems
- Data entry applications

■ Key Characteristics:

- No interaction between user and computer once job starts.
- Jobs are executed in the order they arrive (FIFO), though priority scheduling was later added.
- Once a batch is loaded, jobs execute automatically
- Used to communicate with the OS and describe jobs and resources.

■ How It Works

### Job Submission

Users write jobs on punched cards or magnetic tape and submit them to the operator.

### Batching

The operator groups jobs into batches with similar requirements.

### Job Execution

The OS reads one job at a time from the batch and executes it.

### Result Storage

Output is stored or printed for user review after the batch completes.

College Sahayak

■ Advantages

### Efficient for large volume tasks
Suitable for repetitive jobs like payroll, billing, or inventory processing.

### Minimized idle time
CPU utilization is higher since jobs are processed without waiting for user input.

### Simplified Management
Jobs are handled in groups, making it easier for operators to manage workloads.

■ Disadvantages

### No real-time interaction
Users must wait until the job is completed to see results or errors.

### Difficult debugging
Errors are discovered only after the batch completes, not during execution.

### Poor for interactive applications
Not suitable for modern GUI-based or real-time tasks.

■ Real-Life Example

### Think of a batch OS like a washing machine:
- You load a batch of dirty clothes.
- Select a program.
- Press start.
- You can't change anything mid-cycle.
- At the end, you get the cleaned clothes.

■ Examples of Batch Operating Systems

**IBM OS/360**

**SCOPE (for CDC systems)**

**GEOS (used in General Electric computers)**

### Multi-programmed Operating System
- Multiprogramming means more than one program can be active at the same time.
- Before the operating system concept, only one program was to be loaded at a time and run.
- These systems were not efficient as the CPU was not used efficiently.
- For example, in a single-tasking system, the CPU is not used if the current program waits for some input/output to finish.
- The idea of multiprogramming is to assign CPUs to other processes while the current process might not be finished.
- All modern operating systems like MS Windows, Linux, etc are multiprogramming operating systems.

### Features of Multiprogramming:
- Only one CPU is needed to run it.
- The system can switch between different programs (processes).
- It switches when a program has to wait (like for input/output).
- The CPU is kept busy most of the time.
- Computer resources (like CPU, memory) are used more efficiently.
- Overall performance is better.

### How Multiprogramming Operating Systems Work?

College Sahayak

- In a multiprogramming system, several programs are kept in memory at the same time.

**Each program is called a process, and the operating system manages all of them.**

**The operating system chooses one ready process to run on the CPU.**

- If that process needs to do an input/output (I/O) task (like reading a file), it stops using the CPU and is moved to storage.
- Then, the CPU switches to another ready process.
- Once the I/O task is done, the original process can return to the CPU.
- This switching happens very quickly, so it looks like all programs are running at the same time.

### Advantages of Multiprogramming:

- Increase the resource utilization of a computer system.
- Support multiple simultaneously interactive users (terminals).
- Keeps multiple runnable jobs loaded in memory.
- Increase the throughput of the system.
- The waiting time for a program is reduced.
- Improve system's overall responsiveness.
- Improve the reliability and stability of the system.
- Make the system suitable for multitasking environments.

### Disadvantages of Multiprogramming:

- You need to understand how scheduling works to manage which task runs next.
- When there are too many jobs, some might have to wait a long time.
- The system must manage memory well since many tasks are stored there.
- Running too many programs at once can cause the system to overheat.

### Examples of Multiprogramming Operating Systems

- IBM OS/360
- UNIX
- Windows NT
- Linux
- macOS

### Time Sharing Operating System

- A Time-Shared Operating System (TSOS) is a type of multiprogramming operating system designed to allow multiple users to share system resources (like CPU, memory, I/O devices) simultaneously or concurrently.
- It is also known as a time-sharing system.
- Time-sharing means dividing the CPU time among multiple users or tasks.
- A time-shared operating system uses CPU scheduling and multi-programming to provide each user with a small portion of a shared computer at once.
- Each user has at least one separate program in memory.
- A program is loaded into memory and executes, it performs a short period of time either before completion or to complete I/O.
- This short period of time during which the user gets the attention of the CPU is known as time slice, time slot, or quantum.
- Time-shared operating systems are more complex than multi-programmed operating systems.
- In both, multiple jobs must be kept in memory simultaneously, so the system must have memory management and security.
- In the above figure the user 5 is active state but user 1, user 2, user 3, and user 4 are in a waiting state whereas user 6 is in a ready state.
- Active State - The user's program is under the control of the CPU. Only one program is available in this state.

College Sahayak

- Ready State - The user program is ready to execute but it is waiting for its turn to get the CPU. More than one user can be in a ready state at a time.
- Waiting State - The user's program is waiting for some input/output operation. More than one user can be in a waiting state at a time.

### Advantages

- Each task gets an equal opportunity.
- Fewer chances of duplication of software.
- CPU idle time can be reduced.

### Disadvantages

- Reliability problem.
- One must have to take of the security and integrity of user programs and data.
- Data communication problem.

### Multiprocessor Operating System

- A multiprocessing operating system uses two or more CPUs to make the system run faster.
- These processors work at the same time to complete tasks.
- They are all connected to memory, devices, and other system parts.
- The main goal is to speed up how quickly the system works.
- This improves the system’s overall performance. Examples of such systems include UNIX, LINUX, and Solaris.
- A multi-processing operating system has more than one CPU.
- Each CPU is linked to the main memory.
- The work is divided among all the CPUs.
- To run faster and perform better, each CPU gets its own task.
- After all CPUs finish their tasks, the results are combined into one output.
- The operating system manages the resources for each CPU.
- This leads to better use of resources and improved performance.
- The below diagram describes the working of multi-processing operating systems.
- Types of Multi-Processing Operating Systems

#### Symmetrical Multiprocessing Operating System

#### Asymmetrical Multiprocessing Operating System

#### Symmetrical Multiprocessing Operating System

- In a Symmetrical Multiprocessing (SMP) operating system, all processors run the same copy of the operating system.
- Each processor makes its own decisions and works with the others to keep the system running smoothly.
- CPU scheduling algorithms help assign tasks to the processor with the least load. SMP is also called a "Shared Everything System" because all processors share memory and input-output resources.
- The image below shows how a symmetrical multiprocessing system works.

### Advantages:

- If one processor fails, the others keep working.
- The workload is shared evenly among all processors.
- It uses system resources efficiently.

### Disadvantages:

- Symmetrical multiprocessing systems are more complex.
- They are more expensive.
- It is hard to keep all processors properly synchronized.

### Asymmetrical Multiprocessing Operating System

College Sahayak

- In an Asymmetrical Multiprocessing (AMP) operating system, one processor is the master, and the others are slaves.
- The master assigns tasks to the slave processors.
- It keeps a ready queue of processes for the slaves to run.
- The master also creates a scheduler to manage and assign these tasks.
- The diagram below shows how an asymmetrical multiprocessing system works.

### Advantages:

- Asymmetrical multiprocessing systems are affordable.
- They are simple to design and handle.
- They can be expanded easily.

### Disadvantages:

- Work may not be shared equally between processors.
- Processors do not use the same memory.
- If one process fails, the whole system can stop working.

### Distributed Operating System

- A Distributed Operating System uses many CPUs but looks like a single, normal system to users.
- It lets different resources like CPUs, disks, networks, and computers be shared across various locations, increasing the available data.
- All processors are connected through fast links such as buses and telephone lines, and each has its own local memory.
- Because of this, a distributed system is called loosely connected. It includes many computers and sites linked by LAN or WAN networks.
- It lets users share processing power and files while working like one virtual machine.
- The diagram below shows how a distributed operating system is set up.

### Types of Distributed Operating Systems

#### Client-Server System

In this system, the client asks for resources, and the server provides them. A server can serve many clients at the same time. This is also called a Tightly Coupled OS, mainly used in multiprocessors and similar computers. The server handles all client requests centrally.

- Computer Server System: The client sends requests to the server, which performs the action and sends back the result.
- File Server System: The server manages files for clients, allowing them to create, delete, or update files.

### Peer-to-Peer System

Here, all computers (nodes) share tasks equally and can share resources and data as needed. They connect through a network. This system is called Loosely Coupled because each computer has its own memory and communicates over networks like telephone lines or high-speed buses.

### Middleware

Middleware helps programs running on different operating systems communicate with each other easily by sharing information.

### Three-tier System

In this setup, data from the client is handled by an intermediate layer, not directly by the client. This makes development easier and is common in online applications.

### N-tier System

Used when a server or app needs to send requests to other services over a network, involving multiple layers (or tiers) of processing.

### Examples of Distributed Operating Systems

**Solaris: Made for SUN multiprocessor workstations.**

**OSF/1: Created by the Open Software Foundation and works with Unix.**

**Micros: Assigns tasks to all nodes and keeps the data load balanced.**

**DYNIX: Designed for computers with many processors (called Symmetry).**

- Locus: Lets users access files locally or from far away without any location limits.

**Mach: Supports multitasking and running many threads at the same time.**

### Real-Time Operating System

- A Real-Time Operating System (RTOS) is a special kind of operating system made to complete tasks quickly and on time.
- Unlike regular operating systems that focus on multitasking and user interaction, an RTOS focuses on doing things right when they need to be done.
- The idea of real-time computing started a long time ago. The first RTOS was made by Cambridge University in the 1960s.
- It allowed many processes to run at once but made sure each finished within a set time.
- Since then, RTOS has improved a lot. Today, these systems are faster, more reliable, and have many features. They are used in fields like aerospace, defense, medical devices, multimedia, and more.

### Types of Real-Time Operating Systems (RTOS):

**Hard Real-Time OS**

- Tasks must be completed within strict deadlines.
- Missing a deadline can cause serious problems (e.g., in medical or aerospace systems).

### Soft Real-Time OS

- Tasks should finish on time, but missing deadlines is not critical.
- It can handle delays without major issues (e.g., video streaming or online gaming).

### Firm Real-Time OS

- Deadlines are important, but missing them occasionally is acceptable.
- If a deadline is missed, the task's result is useless but won't cause system failure.

### Advantages of RTOS:

- Fast and predictable response: RTOS guarantees tasks finish on time, which is crucial for critical systems.

**Reliable: Designed to handle important jobs without failure.**

**Multitasking: Can run many tasks simultaneously with precise timing.**

**Efficient resource use: Manages CPU and memory carefully to meet deadlines.**

- Useful in critical fields: Perfect for aerospace, medical devices, industrial control, etc.

### Disadvantages of RTOS:

**Complex design: Hard to develop and maintain due to strict timing rules.**

**Cost: Usually more expensive than general-purpose operating systems.**

- Limited user interface: Often has simpler interfaces since it focuses on real-time tasks.

**Less flexibility: Optimized for specific tasks, not general computing.**

- Resource constraints: May need special hardware to support real-time capabilities.

College Sahayak

### Mobile Operating System

- Mobile operating systems are made specifically for devices like smartphones and tablets.
- Examples include Android and iOS.
- They manage the device's hardware and software and let users run apps smoothly.

### Advantages:

**Easy to Use: Designed to be simple and user-friendly for everyone.**

**Lots of Apps: Huge variety of apps lets users customize their devices.**

**Stay Connected: Support many ways to connect to the internet and other devices.**

**Regular Updates: Get frequent updates with new features and security fixes.**

### Disadvantages:

**Battery Life: Batteries can run out quickly, especially with heavy use.**

- Security Risks: Mobile devices can be attacked by malware or phishing, risking user data.
- Fragmentation: Especially in Android, many device types make it hard for developers to make apps that work everywhere.
- Limited Resources: Mobile devices have less power, memory, and storage than computers, which can slow down demanding apps.

---

## Command line based OS – DOS:

- Operating systems can come with either a command-line interface (CLI) or a graphical user interface (GUI), or sometimes both.
- DOS was a command-line operating system that was widely used in the 1980s and 1990s.
- MS-DOS (Microsoft DOS) is probably the most well-known version.
- DOS did not have a graphical user interface, instead, users had to type commands to perform tasks like navigating the file system or launching applications.

### Advantages of MS-DOS Operating System:

- It is a minimalist OS which means it can boot a computer and run programs.
- Still usable for simple tasks like word processing and playing games.
- The mouse cannot be used to give inputs instead it uses basic system commands to perform the task.
- It is a 16-bit, free operating system.
- It is a single-user operating system.
- It is very lightweight due to fewer features available and no multitasking.

### Disadvantages of MS DOS Operating System:

- It is not a multitasking operating system that is we cannot run too many applications in the background.
- Files on the system can be easily deleted or the system can also be easily destroyed.
- It does not provide any warning message before you delete or perform any unwanted task like in windows or Linux.
- It is text-based and it does not have any graphical user interface.
- Not secure to be used in any kind of public network.
- Encryption is not supported.
- Difficulty in memory access.
- Mouse cannot be used to give inputs.

---

## Command line based OS – UNIX:

- UNIX is a powerful, multiuser, multitasking command-line based operating system originally developed in the 1970s at AT&T;'s Bell Labs.

College Sahayak (c) 2026 | Verified MSBTE Study Material | All Rights Reserved

- It is known for its stability, security, and flexibility, and it forms the foundation for many modern operating systems like Linux, macOS, and Android.
- UNIX has both command-line and graphical versions today, it was originally designed and operated through the command line interface (CLI).

■ Key Features of UNIX

**Multi-user system: Multiple users can work on the same system at the same time.**

**Multitasking: Can run many programs (processes) at the same time.**

**Portability: UNIX can be easily adapted to work on different types of hardware.**

**Security: Strong user authentication and file permission system.**

- Modular design: Made up of small utilities that can be combined to perform complex tasks.
- Command-line based: Users interact with the system using typed commands in a shell.

■ Main Components of UNIX:

### Kernel

- The core of the OS.
- Manages hardware, memory, processes, and system calls.

### Shell

- The command-line interface that interacts with the user.
- Translates user commands into instructions the kernel can understand.

### File System

- Hierarchical structure starting from the root directory (/).
- Everything (including devices and programs) is treated as a file.

### Utilities/Commands

- A set of standard programs that allow users to perform tasks like file manipulation, program execution, and text processing.

### Advantages of UNIX Operating System:

- Multiuser Support
- Multiple users can log in and use the system at the same time without interfering with each other.
- Ideal for servers and shared environments.
- Multitasking
- UNIX can run many programs simultaneously.
- It handles background processes efficiently.
- Security and Permissions
- UNIX provides strong user authentication and file permission controls.
- Files can have different access levels for the owner, group, and others.
- Portability
- UNIX is written mostly in C, which makes it easy to modify and move to different hardware platforms.
- Stability and Reliability
- UNIX systems rarely crash and can run for long periods without needing a reboot.
- Frequently used in critical systems (like web servers and scientific environments).
- Modularity
- UNIX uses small, specialized tools that can be combined to perform complex tasks using pipes and redirection.
- Powerful Shell and Scripting
- The UNIX shell allows users to automate tasks and write powerful scripts for system administration and development.
- Large Community and Documentation

College Sahayak

• Being around for decades, UNIX has extensive documentation and a strong developer community.

### Disadvantages of UNIX Operating System

• Complex Command Line Interface
• For beginners, UNIX can be hard to learn because of its command-line nature.
• A steep learning curve compared to GUI-based systems like Windows.
• Lack of Standardization
• Different UNIX versions (e.g., AIX, Solaris, HP-UX) may have variations in commands and behaviour.
• Software Compatibility
• Many commercial applications (like Microsoft Office or Adobe products) are not available for UNIX.
• Limited support for gaming or entertainment software.
• Hardware Support
• Some specialized or modern hardware may not have drivers or support in certain UNIX systems.
• GUI Limitations
• Although modern UNIX variants can support a graphical user interface, it's not as user-friendly or consistent as Windows or macOS.
• GUI setups often require manual configuration.
• Cost (for some versions)
• Commercial UNIX versions like IBM AIX or Oracle Solaris can be expensive.

---

## GUI based Operating System - WINDOWS:

• A GUI-based operating system is an operating system that lets users interact with the computer using graphics such as icons, buttons, windows, menus, and the mouse, instead of typing commands in a text interface.
• It's the most common type of OS used today, especially in personal computers, tablets, and smartphones.

### What is Windows?

**Windows is a GUI-based (Graphical User Interface) operating system developed by**

### Microsoft.

• Windows allows users to interact with the computer using visual elements like windows, icons, buttons, and menus, instead of typing text commands.
• The first version, Windows 1.0, was released in 1985, and since then it has evolved into one of the most widely used operating systems in the world (e.g., Windows 10, Windows 11).

### Key Features of Windows Operating System

• Graphical User Interface (GUI)
• Users interact with the system using icons, buttons, menus, and windows.
• Easier to learn and use than text-based systems.
• Multitasking
• Can run multiple applications at the same time (e.g., using Word while browsing the internet and listening to music).
• Multi-user Support
• Allows multiple users to create separate accounts on the same computer, each with its own settings and files.
• Device Management
• Automatically detects and installs drivers for hardware like printers, USB drives, and cameras.
• File Management
• Provides a file explorer to easily browse, create, copy, move, rename, and delete files and folders.
• Security Features
• Includes antivirus protection, user account control (UAC), firewall, encryption, and system updates.
• Plug and Play

College Sahayak

- Automatically recognizes new hardware and installs drivers without manual setup.
- Networking Capabilities
- Built-in support for local networks, internet access, and sharing files/devices across multiple systems.
- Help and Support
- Built-in Help Center and troubleshooting tools to assist users with common problems.

### Advantages of Windows

- User-friendly interface
- Supports a wide range of software and hardware
- Regular updates and patches
- Built-in tools for maintenance and recovery
- Widely used and supported globally

### Disadvantages of Windows

- Can be resource-heavy (requires more RAM and CPU)
- Prone to viruses and malware if not properly protected
- Paid license (not free like some Linux distributions)
- Less customizable compared to open-source operating systems

---

## GUI based Operating System - LINUX:

### What is Linux?

- Linux is an open-source, multiuser, multitasking operating system based on UNIX.
- Linux started as a command-line-based OS, modern Linux distributions include Graphical User Interfaces (GUIs), making them much more user-friendly and accessible even for beginners.
- Today, Linux is used in a wide range of devices including PCs, laptops, servers, mobile phones, smart TVs, routers, and supercomputers.
- A GUI in Linux allows users to interact with the system using icons, buttons, windows, and menus, rather than typing commands.

### Key Features of GUI-Based Linux

### Open Source & Free

- You can download, modify, and share Linux for free.

### Customizable Interface

- You can choose your desktop environment and change themes, panels, icons, and more.

### Multitasking

- Run multiple apps at once using tabs, workspaces, and virtual desktops.

### Security & Privacy

- Linux is less vulnerable to viruses and provides strong file permissions and system security.

### Package Managers

- Easily install or update software using GUI-based app stores

### Multiple Desktops

- Users can switch between desktops or sessions, helpful in shared environments.

### Advantages of GUI-Based Linux

**User-Friendly: GUI makes Linux accessible for all users.**

**Free and Open Source: No license cost.**

College Sahayak

**Highly Customizable: Choose from many GUIs and themes.**

**Secure: Fewer viruses and malware than Windows.**

**Community Support: Large online community, forums, and documentation.**

**Lightweight Options: Can run on older or low-end computers.**

### Disadvantages of GUI-Based Linux

**Software Compatibility: Some popular Windows programs may not run directly.**

**Gaming Support: Improving, but still not as strong as Windows.**

**Hardware Drivers: Some devices may lack proper drivers out of the box.**

**Learning Curve: Users may need time to get used to the layout and features.**

---

## GUI based Operating System - macOS:

### What is macOS?

- macOS (formerly known as Mac OS) is a GUI-based (Graphical User Interface) operating system developed by Apple Inc.
- It is designed to run exclusively on Apple computers, such as the MacBook, iMac, Mac Mini, and Mac Studio.
- macOS offers a user-friendly, secure, and visually appealing environment for users.
- It is known for its smooth performance, tight integration with Apple hardware, and seamless ecosystem with other Apple devices like iPhones and iPads.

### Key Features of macOS as a GUI-Based OS

- Graphical User Interface (GUI)
- Features a clean, elegant, and intuitive GUI.
- Users interact using icons, windows, menus, toolbars, and drag-and-drop functionality.
- Smooth animations and transitions enhance user experience.
- Dock
- A bar typically at the bottom of the screen that provides quick access to frequently used applications, folders, and running apps.
- Finder
- macOS's built-in file manager. Helps users to browse, search, move, and organize files and folders visually.
- Menu Bar
- Located at the top of the screen. Provides access to system menus, app controls, Wi-Fi, battery, sound, time, and notifications.
- System Preferences / Settings
- GUI-based interface for changing system settings like display, sound, users, network, and privacy.
- Mission Control
- A view that shows all open windows, spaces (virtual desktops), and apps to manage multitasking visually.
- Spotlight Search
- A fast search tool that lets users find files, apps, web results, and more using a single text box.
- Advantages of macOS

**User-Friendly GUI: Clean, smooth, and consistent user experience.**

- Tightly Integrated with Apple Hardware: Optimized for performance, speed, and battery life.
- Security & Stability: Fewer viruses, strong sandboxing, and Unix-based security model.
- Excellent Display & Graphics: Retina display support and graphic-intensive design tools.
- Seamless Apple Ecosystem: Works perfectly with iPhone, iPad, Apple Watch, and iCloud.
- Pre-installed Software: Comes with powerful built-in apps like Safari, Photos, GarageBand, iMovie, Pages, and Keynote.

College Sahayak

• Disadvantages of macOS
• Expensive Hardware: Only runs on Apple devices, which are often costlier than Windows/Linux systems.

**Less Customizable: Limited customization compared to Linux.**
• Software Compatibility: Some software (especially gaming or enterprise tools) may not be available.

**Limited for Gamers: macOS is not widely supported for high-end gaming.**
• Closed Ecosystem: Not as open as Linux; restricted to Apple-approved tools and hardware.

---

## Different Services of Operating System:

• Program Execution

The OS is responsible for loading programs into memory and starting their execution. It manages the running programs (processes) by allocating CPU time and system resources until the program finishes. It also handles process creation, suspension, and termination.

• File System Management

The OS organizes data in files and directories (folders) on storage devices. It handles creating, reading, writing, and deleting files, and manages file permissions to control who can access or modify the data. The OS also keeps track of where files are stored on the disk and maintains the integrity of the file system.

• Input/output (I/O) Management

This service manages communication between the OS and hardware devices like keyboards, mice, printers, disk drives, and display screens. It uses device drivers to abstract hardware details, handles buffering and caching to improve speed, and schedules I/O requests to optimize performance.

• Resource Allocation

The OS manages all hardware resources such as CPU time, memory space, and input/output devices. When multiple programs or users request resources, the OS decides how to allocate them efficiently and fairly, ensuring no program is starved of resources and that the system runs smoothly.

• Error Detection and Handling

The OS constantly monitors the system to detect errors in hardware (like memory failures or disk errors) and software (such as invalid instructions). When errors occur, the OS takes appropriate action — such as terminating faulty processes, notifying users, or attempting recovery — to maintain system stability.

• User Interface

The OS provides an interface that allows users to interact with the computer. This can be a command-line interface (CLI) where users type commands, or a graphical user interface (GUI) with windows, icons, and menus, making it easier to operate the system.

• Security and Access Control

The OS protects data and system resources by enforcing security policies. It authenticates users (e.g., through passwords), controls access rights to files and devices, and prevents unauthorized access to sensitive information or system functions.

• Process Synchronization and Communication

When multiple processes run concurrently, the OS coordinates their actions to prevent conflicts, especially when they share resources or data. It provides synchronization tools (like semaphores and mutexes) and communication methods (like message passing or shared memory) to enable safe and efficient process interaction.

• System Performance Monitoring

The OS tracks how well the system is performing by monitoring CPU usage, memory consumption, disk activity, and network traffic. This helps detect bottlenecks, optimize resource use, and provide feedback for system tuning or troubleshooting.

• Accounting

College Sahayak

The OS records details about resource usage by users and processes. This information can be used for billing purposes, system auditing, or to analyse usage patterns, helping manage and plan system capacity.

## System Calls: Concept, types of system calls:

- A system call is a way for a program to request a service from the operating system's kernel.
- Applications cannot directly access hardware or perform operations for security and stability reasons, they must use system calls to interact with the OS.
- System calls as a bridge between user programs and the operating system.
- When a program needs to do something like read a file, write to the screen, or allocate memory, it asks the OS via a system call.

Types of System Calls

System calls are generally grouped into several categories based on the services they provide:

- Process Control System Calls

These manage processes - programs in execution. Examples include:

**create process: Start a new process.**

**terminate process: End a running process.**

**wait: Wait for a process to finish.**

**execute: Run a program.**

**get process attributes: Obtain info like process ID.**

- File Management System Calls

Used to create, open, read, write, and delete files.

**create: Make a new file.**

**open: Open an existing file.**

**read: Read data from a file.**

**write: Write data to a file.**

**close: Close a file.**

**delete: Remove a file.**

- Device Management System Calls

Handle communication with hardware devices.

**request device: Request control of a device.**

**release device: Release a device after use.**

**read device: Read data from a device.**

**write device: Send data to a device.**

**get device attributes: Get device status info.**

- Information Maintenance System Calls

Provide system and process information.

**get system time: Obtain the current time.**

**set system time: Change the system clock.**

**get process ID: Retrieve the ID of the current process.**

**get system data: Obtain system-related statistics or info.**

College Sahayak

• Communication System Calls

Support communication between processes, especially in distributed or multi-process systems.

**create communication connection: Set up a communication link.**

**send message: Send data to another process.**

**receive message: Receive data from another process.**

**close communication: End a communication link.**

---

## Operating System Components: Process Management, Main Memory Management, File Management, IO Management, Secondary Storage Management:

### Operating system components

- An operating system (OS) is an important software layer that manages computer hardware and software resources.
- It providing a platform for applications to run and users to interact with the system.
- It is combined of several interconnected components working together to achieve efficient and reliable system operation.

### Here are the key components of an operating system:

### Process management

- Process management controls how computer programs (called processes) run.
- A process is just a program that is currently running, and it has its own memory and resources.

### Key functions:

- Process creation and deletion: The OS can start a new process when needed and remove it once it’s done or no longer needed.
- Process scheduling: The OS decides which process gets to use the CPU and for how long. This helps make sure the CPU is used efficiently and all processes get a fair chance. Common methods include:

**First-Come, First-Served (FCFS): Processes run in the order they arrive.**

**Shortest Job Next (SJN): Shorter tasks go first.**

**Round Robin (RR): Each process gets equal time in turns.**

**Priority Scheduling: Higher priority processes run first.**

- Multilevel Queue: Processes are grouped and scheduled based on their type or priority.
- Process synchronization: When processes share data or resources, the OS makes sure they don’t interfere with each other. Tools like semaphores, mutexes, and monitors help keep data safe and avoid errors.
- Inter-process communication (IPC): The OS provides ways for processes to talk to each other and share data. This can be done using shared memory, messages, or pipes.
- Context switching: The OS saves the current state of one process and loads another so multiple processes can run as if they’re happening at the same time.

### Main memory management

- Main memory, or RAM, is where the computer stores programs and data that are currently being used.
- The Operating System manages this memory to make sure it’s used efficiently and safely.

College Sahayak

# UNIT II Process Management

*CO2: Describe the different aspects of Process Management in an OS. Total Marks: 14*

**Syllabus: Total Marks:14**

## Introduction to Process Management:

- A process is a running instance of a program. It is the smallest unit of work that can be scheduled and executed by the operating system (OS).
- To perform its tasks, a process needs essential resources like:

**CPU time**

**Memory**

**Files**

**I/O devices**

- The operating system uses processes to manage the execution of both sequential and concurrent programs efficiently.

**Process management involves:**

- Allocating resources to processes.
- Allowing processes to share and exchange information.
- Protecting process resources from interference by other processes.
- Enabling synchronization among processes.
- OS maintains control over each process using internal data structures that track process state, resource ownership, and other critical data.

## Process:

**A process can be defined as:**

- An entity representing the basic unit of work implemented in a system.

**A program under execution that competes for CPU time and other resources.**

**Often referred to as a job, task, or unit of work.**

- Processes progress sequentially, meaning at any given time, only one instruction from the process executes.
- In memory, a process consists of:

**Text Section: Contains the program code.**

**Data Section: Stores global and static variables.**

- Heap: Manages dynamic memory allocation via functions like malloc, new, and delete.
- Stack: Stores local variables, subroutine parameters, return addresses, and temporary data.
- Program Counter: Contains the address of the current instruction being executed.
- Multiprogramming allows multiple processes to exist simultaneously within the memory.

**However, at any single moment, only one process can execute on the CPU.**

- The CPU rapidly switches between processes using context switching, giving the illusion that multiple processes are running in parallel

College Sahayak

## Difference between Program & Process:

## Process States in Operating System:

A process goes through multiple states from its creation to termination. These states help the operating system manage processes effectively.

• New State

**This is the initial state of a process.**

**The process is being created by the operating system.**

• Necessary resources are allocated, but the process has not yet entered the ready queue for execution.
• Example: When you start an application, it enters the New state first.
• Ready State
• In this state, the process is ready to execute and is waiting in the queue for the CPU to be assigned.

**It has all required resources except the CPU.**

• It waits its turn as per the scheduling algorithm used by the OS (like FCFS, Round Robin, etc.).
• Multiple processes may be in Ready state at the same time.
• Running State

**The process is in the active execution phase.**

**The CPU is currently executing the instructions of this process.**

• During this state, the process performs its computations and may also produce outputs.
• Note: At any single instant, only one process can be in Running state on a single-core CPU.
• Waiting or Blocked State
• The process enters this state when it requires an input/output operation or is waiting for an

**event to occur.**

• It cannot continue execution until the event or I/O completes.
• While waiting, the process is not consuming CPU time, and is kept aside until it's ready to proceed.
• Example: Waiting for user input, file read/write, etc.
• Terminated State

**Also called the Completed or Exit State.**

• After successful execution or due to errors/failure, the process moves to Terminated state.
• The operating system frees up resources allocated to the process like memory, files, etc.

**The process is now considered dead and is removed from the process table.**

---

## Process Control Block:

• Process Control Block is a data structure that contains information of the process related to it.
• The process control block is also known as a task control block, entry of the process table, etc.
• It is very important for process management as the data structuring for processes is done in terms of the PCB.
• It also defines the current state of the operating system.
• Structure of the Process Control Block: The process control stores many data items that are needed for efficient process management. Some of these data items are explained with the help of the given diagram.
• Process State: This specifies the process state i.e. new, ready, running, waiting or terminated.

**Process Number: This shows the number of the particular process.**

• Program Counter This contains the address of the next instruction that needs to be executed in the process.
• Registers: This specifies the registers that are used by the process. They may include accumulators, index registers, stack pointers, general purpose registers etc.
• List of Open Files: These are the different files that are associated with the process

College Sahayak

- CPU Scheduling Information: The process priority, pointers to scheduling queues etc. is the CPU scheduling information that is contained in the PCB. This may also include any other scheduling parameters.
- Memory Management Information: The memory management information includes the page tables or the segment tables depending on the memory system used. It also contains the value of the base registers, limit registers etc.
- I/O Status Information: This information includes the list of I/O devices used by the process, the list of files etc.
- Accounting information: The time limits, account numbers, amount of CPU used, process numbers etc. are all a part of the PCB accounting information.
- Location of the Process Control Block: The process control block is kept in a memory area that is protected from the normal user access. This is done because it contains important process information. Some of the operating systems place the PCB at the beginning of the kernel stack for the process as it is a safe location.

## Process Scheduling:

- A part of the Operating System that decides which process should run next on the CPU.
- Process Scheduling is the method by which the operating system decides which process will use the CPU next.
- It helps in efficient use of the CPU and improves system performance.

### Why is Process Scheduling Important?

- Keeps CPU busy all the time.
- Reduces waiting time for processes.
- Provides fair chances to all processes.
- Improves system response time.
- Increases overall system throughput (number of processes completed).

## Scheduling Queues:

- All PCBs (Process Scheduling Blocks) are kept in Process Scheduling Queues by the OS.
- Each processing state has its own queue in the OS, and PCBs from all processes in the same execution state are put in the very same queue.
- A process's PCB is unlinked from its present queue and then moved to its next state queue when its status changes.
- The following major process scheduling queues are maintained by the Operating System:
- Job Queue
- Contains all processes in the system.
- As soon as a process is created, it enters the job queue.
- Ready Queue
- Contains all processes that are ready to run and waiting for CPU time.
- Managed by the Short-Term Scheduler.
- Example: When a process is in memory but waiting for CPU.
- Device Queue (I/O Queue)
- Contains processes waiting for I/O devices (like printer, disk).
- If a process requests I/O, it is moved to the device queue.
- Waiting Queue
- Sometimes called Blocked Queue.
- Contains processes waiting for some event to occur (like file download completion).

### Movement Between Queues

## Types of Schedulers:

- Schedulers are computer programmes that manage process scheduling in a variety of ways.

College Sahayak

- Their primary responsibility is to choose which jobs to submit into the system and which processes to run.
- There are three types of schedulers:
- Long-Term Scheduler
- Short-Term Scheduler
- Medium-Term Scheduler

### Long-Term Scheduler (Job Scheduler)

- A job scheduler is another name for it.
- A long-term scheduler determines which programs are accepted for processing into the system.
- It picks processes from the queue and then loads them into memory so they can be executed.
- For CPU scheduling, a process loads into memory.
- The long-term scheduler controls the degree of multiprogramming by deciding which processes are admitted to the system and loaded into memory for execution.
- To maintain a balance between CPU-bound and I/O-bound processes, ensuring efficient resource utilization.
- It operates relatively slowly compared to other schedulers as it deals with the overall system load.
- Example: It select processes from a job queue on disk and load them into the ready queue in memory.

### Short-Term Scheduler (CPU Scheduler)

- CPU scheduler is another name for it.
- Its major goal is to improve system performance according to the set of criteria defined.
- It refers to the transition from the process's ready state to the running stage.
- The CPU scheduler happens to choose a process from those that are ready to run and then allocates the CPU to it.
- The short-term scheduler selects which process from the ready queue will be allocated the CPU next.
- It helps to maximize CPU utilization and minimize response time.
- It operates very frequently and quickly, making decisions in milliseconds.
- Example: When a process completes its time slice or is blocked for I/O, the short-term scheduler chooses the next process to run.

### Medium-Term Scheduler

- The medium-term scheduler is responsible for swapping processes in and out of main memory to manage the degree of multiprogramming and improve system performance.
- It can temporarily remove processes from memory (swap them out) to free up resources and bring them back later (swap them in) when resources are available.
- It helps to handle situations where the system has too many processes in memory, causing performance degradation.
- Example: If the system is overloaded, the medium-term scheduler might swap out some processes to secondary storage, allowing other processes to run.

---

## Difference between schedulers:

## Context Switch:

- Context Switching is the process where the CPU switches from running one process to another.
- It saves the current process's state and loads the next process's state.
- Why Context Switching Occurs?
- When a process is blocked (waiting for I/O).
- Time slice expires in pre-emptive scheduling (e.g., Round Robin).
- A higher priority process arrives.
- Voluntary yield by the process itself.
- Following Steps done in Context Switching

**Pause the current process.**

- Save its state (PC, registers) into its PCB.
- Choose the next process using the scheduler.
- Load the next process's state from its PCB.
- Resume execution of the new process.
- What is Saved During Context Switching?

**Program Counter (PC) – remembers where the process stopped.**

**CPU Registers – holds the current process's working data.**

**Process State – stored in Process Control Block (PCB).**

- Advantages of Context Switching

**Supports multitasking.**

**Improves CPU utilization.**

- Allows responsive systems (important for real-time OS).
- Disadvantages of Context Switching
- Adds CPU overhead (no productive work during switching).
- Too many switches reduce overall system performance.
- Complex to manage efficiently in large systems.

---

## Inter Process Communication (IPC):

- IPC stands for Inter Process Communication.
- It is the method by which processes exchange data and messages with each other.
- Necessary in multitasking systems where processes need to work together.
- Two main methods of IPC: Shared Memory and Message Passing.
- Why is IPC Needed?
- To share data between processes.
- For synchronization of processes.
- To coordinate actions between different processes.
- To avoid race conditions.
- Example: Uses of IPC
- Chat applications (processes exchanging messages).
- Web servers (handling multiple clients).
- File sharing between different applications.
- Advantages of IPC
- Enables process cooperation.
- Helps in efficient resource sharing.
- Supports modular and distributed systems.
- Disadvantages of IPC
- Can cause synchronization problems (race conditions, deadlocks).
- Overhead in managing communication.

---

## Shared Memory System:

- A Shared Memory System is a type of Inter Process Communication (IPC) method.
- In this system, multiple processes share a common memory space to exchange data.
- The Operating System sets up a shared memory region in RAM.
- Different processes can read from and write to this memory area.
- The data is not copied between processes, making communication fast.

College Sahayak

- It gives Direct access to shared data.
- Processes must use synchronization tools to avoid conflicts.
- Used mainly for fast communication between processes running on the same system.
- Shared Memory is the fastest IPC method, but requires careful synchronization to avoid errors.
- Steps in Shared Memory Communication

**Process A writes data into shared memory.**

**Process B reads data from shared memory.**

- Synchronization ensures only one process accesses shared data at a time.
- Advantages of Shared Memory
- Fastest IPC method (no need to copy data).

**Suitable for large data exchange.**

- Direct communication without kernel involvement after setup.
- Disadvantages of Shared Memory
- Risk of data corruption if synchronization is not used.

**Complex to implement due to manual synchronization.**

**Works only for processes running on the same system.**

---

## Message Passing System:

- A Message Passing System is an Inter Process Communication (IPC) method.
- Processes communicate by sending and receiving messages.
- No shared memory is required.
- Each process has a communication channel (like message queue, pipe, or socket).
- Messages are sent through these channels.
- Messages can include simple data or complex objects.

### Types of Message Passing

- Direct Communication

**Processes send messages directly to each other.**

- Requires processes to know each other's identities.
- Indirect Communication

**Messages are sent via a message queue or mailbox.**

- Processes don’t need to know each other.
- Message Passing Methods

**Send() Process sends a message.**

**Receive() Process waits to receive a message.**

- Types of Message Communication

**Synchronous Sender waits until receiver gets the message.**

**Asynchronous Sender sends message and continues without waiting.**

- Advantages of Message Passing
- Simple to implement (compared to shared memory).
- Suitable for distributed systems (processes on different machines).
- Less risk of data corruption (no shared memory).
- Processes don’t need to share address space.
- Disadvantages of Message Passing

College Sahayak

- Slower than shared memory (data must be copied).
- Communication overhead due to message management.
- Managing message queues can become complex.

---

## Threads:

- Thread is the smallest unit of execution within a process.
- It is also called a lightweight process.
- Multiple threads can exist inside a single process and share resources.
- A process can have multiple threads, but every thread belongs to only one process.
- If the system supports multithreading, one process can run many threads.
- Threads work best when there is more than one CPU.
- Threads of the same process share:
- Code
- Data
- File system
- Each thread has its own:
- Stack
- Program Counter (PC)
- Registers
- Why Use Threads?
- Faster Execution – Threads are lighter than processes; context switching is quicker.
- Efficient Resource Sharing – Threads share memory, so they communicate faster than processes.

**Multitasking – Multiple threads can run at the same time (parallel execution).**

- Better Use of Multi-core CPUs – Threads can run on different cores for improved performance.
- Advantages of Threads

### Faster Execution

**Threads run parts of a program simultaneously, improving performance.**

### Efficient CPU Usage

**On multi-core systems, threads can run in parallel on different CPUs.**

### Less Resource Usage

**Threads share memory of the parent process → need less memory than processes.**

### Faster Creation

**Threads are easier and quicker to create and manage than full processes.**

### Improved Application Responsiveness

**Applications stay responsive (don’t freeze) while performing background tasks.**

- Example: You can type in Word while it auto saves in another thread.

### Simplified Communication

- Threads of the same process share data → no complex IPC (inter-process communication) needed.

### Better for Concurrent Programming

- Threads allow tasks like input, output, and computation to happen at the same time.

### Low Context Switching Time

**Switching between threads is quicker than switching between processes.**

College Sahayak

### Scalability

**Threads help applications scale better on multi-core systems.**

- Cost-effective - Requires less overhead (CPU, memory, resources) than using multiple processes.

### Difference between Process & Threads:

### Benefits of Threads:

### Responsiveness

- Threads help applications stay active even if one part is busy or stuck.
- Example: A web browser can load videos (one thread) while still allowing you to scroll and type (another thread).
- A server can keep listening for new requests while one thread handles the current one.

### Resource Sharing

- Threads of the same process automatically share memory and data.
- No need for complex sharing methods like in processes.
- Makes it easier to manage multiple tasks using the same data.

### Economy

**Creating a thread is faster and cheaper than creating a process.**

- Threads use less memory and resources because they share the same memory space.
- Example: In Solaris OS, creating a process is 30x slower than creating a thread.

### Scalability

**On systems with multiple CPUs, different threads can run in parallel.**

- A single-threaded process cannot use multiple CPUs effectively.

**Multithreading increases performance and parallelism.**

### Better Communication System

- Threads can share data easily within the same address space.
- Thread synchronization makes communication safe and fast.
- Useful when large amounts of data need to be shared between tasks.

### Microprocessor Architecture Utilization

- Threads can run on different processors at the same time.
- On single-CPU systems, the CPU quickly switches between threads to create the feel of parallelism.
- Improves use of the processor's full capability.

### Minimized System Resource Usage

- Threads use fewer system resources than full processes.
- They are easier to create, manage, and terminate.

### Enhanced Concurrency

**Threads increase concurrency — multiple operations can occur at the same time.**

- Each thread can run on a different processor in multi-CPU systems.

### Reduced Context Switching Time

- Switching between threads is faster than between processes.
- This is because threads use the same memory space, so less data needs to be saved and restored.

### User Threads:

- User threads are implemented by user-level libraries.
- The operating System doesn't recognize user-level threads directly.

College Sahayak

- Implementation of User threads is easy.
- Context switch time is less.
- No hardware support is required for context switching.
- If one user-level thread performs a blocking operation, then the entire process will be blocked.
- Multithreaded applications cannot take full advantage of multiprocessing.
- User-level threads can be created and managed more quickly.
- Any operating system can support user-level threads.
- Simple and quick to create, more portable, does not require kernel mode privileges for context switching.
- In user-level threads, each thread has its own stack, but they share the same address space.
- User-level threads are more portable than kernel-level threads.

### Advantages:

- User level thread can run on any operating system.
- User level threads are fast to create and manage.
- User thread library easy to portable.
- Low cost thread operations are possible.
- A user thread does not require modification to operating system.

### Disadvantages:

- Multithreaded applications cannot take advantage of multiprocessing.
- At most, one user level thread can be in operation at one time.
- If user thread is blocked in the kernel, then entire process is blocked.

### Kernel Threads:

- Kernel threads are implemented by Operating System (OS).
- Kernel threads are recognized by Operating System.
- Implementation of Kernel-Level thread is complicated.
- Context switch time is more.
- Hardware support is needed.
- If one kernel thread performs a blocking operation, then another thread can continue execution.
- Kernels can be multithreaded.
- Kernel-level level threads take more time to create and manage.
- Kernel-level threads are operating system-specific.
- The application code doesn't contain thread management code;
- In kernel-level threads have their own stacks and their own separate address spaces, so they are separated from each other.
- Kernel-level threads can be managed independently, so if one thread crashes, it doesn't necessarily affect the others.
- It can access to the system-level features like I/O operations.
- Less portable due to dependence on OS-specific kernel implementations.

### Advantages:

- The kernel is fully aware of kernel-level threads, so the scheduler handles the process better.
- The kernel can still schedule another thread for execution if one thread is blocked.
- This type of thread is suitable for applications that are frequently blocked.
- Multi-processor applications in kernel-level threads can fully utilize multiprocessing to their advantage.

### Disadvantages:

- It is slower to create and inefficient.
- The overhead associated with kernel-level threads requires a thread control block.
- Kernel-level threads are not generic and are specific to the Operating System.

College Sahayak

- Kernel-level threads are not as easy to manage as user-level threads.

## Difference between user-level thread and kernel level thread.

## Multi-threading model are of three types.

### Many to Many Model

- In this model, we have multiple user threads multiplex to same or lesser number of kernel level threads.
- Number of kernel level threads are specific to the machine, advantage of this model is if a user thread is blocked we can schedule others user thread to other kernel thread.
- Thus, System doesn't block if a particular thread is blocked.
- It is the best multi-threading model.

### Advantages:

- Many threads can be created as per user's requirements.
- Multiple kernel can be created.

### Disadvantages:

- True concurrency cannot be achieved.
- Multiple threads of kernel are an overhead for OS.
- Performance is less.

### Many to One Model

- In this model, we have multiple user threads mapped to one kernel thread.
- In this model when a user thread makes a blocking system call entire process blocks.
- As we have only one kernel thread and only one user thread can access kernel at a time, so multiple threads are not able access multiprocessor at the same time.
- The thread management is done on the user level so it is more efficient.

### Advantages:

- Totally portable.
- Performance is better.
- One kernel thread controls multiple user threads.
- Mainly used in language systems, portable libraries.

### Disadvantages:

- Cannot take advantage of parallelism.
- One block thread can block all user threads.

### One to One Model

- In this model, one to one relationship between kernel and user thread.
- In this model multiple thread can run on multiple processor.
- Problem with this model is that creating a user thread requires the corresponding kernel thread.
- As each user thread is connected to different kernel, if any user thread makes a blocking system call, the other user threads won't be blocked.

### Advantages:

- Multiple threads can run parallel.
- Less complication in the processing.
- More concurrency because of multiple threads can run in parallel on multiple CPUs.

### Disadvantages:

- Kernel thread is an overhead.
- It reduces the performance of system.
- Limiting the number of total threads.

College Sahayak

• Thread creation involves light weight process creation.

## Execute process commands like: top, ps, kill, wait, sleep, exit, nice

### top – Real-time Process Viewer

Description:

• Shows live (real-time) information about the system's processes.
• Continuously updates the screen with process info like CPU usage, memory, PID, and user.
• Helps monitor which processes are consuming the most resources.

Key Info Shown:

• PID: Process ID
• %CPU: CPU usage
• %MEM: Memory usage
• COMMAND: Name of the program Example: top
• Displays the system processes in a live updating table.

Use:

• Useful for system administrators to monitor performance.

### ps – Process Status

Description:

• Shows a snapshot of current processes.
• Does not update automatically like top.

Common Options:

• ps: Shows processes of current user.
• ps -e: Shows all processes.
• ps aux: Shows all processes with full details.

Example: ps aux

• Displays a list of all running processes with details.

Use:

• Used to find processes, get PID, or troubleshoot.
• kill – Stop a Process Description:
• Sends a signal to a process to stop it.
• By default, it sends SIGTERM (15) to terminate process politely.
• With -9, sends SIGKILL to forcefully stop a process.

Example:

kill 1234 # Politely stop process with PID 1234 kill -9 1234 # Forcefully kill process

Use:

### Stops programs that are unresponsive or stuck.

• wait – Wait for Background Process Description:
• Pauses script execution until a background process completes.

### Often used in shell scripting.

Example:

sleep 5 & # Run sleep in background

wait $! # Wait until background process finishes Use:

• Used to synchronize tasks in scripts.
• sleep – Delay Execution Description:
• Pauses program or script for a certain amount of time.

College Sahayak

• Time can be given in seconds, minutes, etc.

Example:

sleep 3 # Waits for 3 seconds

Use:

**Used in testing, automation, or adding delays in scripts.**

• exit – Exit Shell or Script Description:
• Used to end a shell session or script.

**Can return a status code:**

• 0: Success
• Non-zero: Error or failure

Example:

exit 0 # Normal successful exit exit 1 # Exit with error

Use:

• Used in shell scripts to indicate success or failure.
• nice – Run Process with Priority Description:
• Starts a process with a priority level.
• Priority is called “niceness”:
• Lower value = higher priority
• Higher value = lower priority
• Range: -20 (high) to 19 (low)
• Only root can set negative (high priority) values.

Example:

nice -n 10 ./heavy_task.sh

Use:

• Used to reduce the priority of resource-heavy tasks.

## Summary Table:

College Sahayak

# UNIT III CPU Scheduling

*CO3: Implement various CPU Scheduling algorithms and evaluate effectiveness. Total Marks: 14*

**Syllabus: Total Marks:16**

## Scheduling:

**-In a computer system, the CPU (Central Processing Unit) is the brain that performs all the calculations and operations.**

-Many processes (programs) want to use the CPU at the same time.

-But the CPU can only work on one process at a time. So, the system needs a way to

**decide which process gets the CPU next.**

**-This decision-making process is called CPU Scheduling.**

**-The main goal of CPU scheduling is to maximize CPU usage, reduce waiting time, and improve system performance. It helps in handling multiple processes efficiently.**

**-There is a special part of the operating system called the CPU Scheduler. It selects one process from the ready queue (list of waiting processes) and assigns the CPU to it.**

## Why Scheduling is Needed

**A computer runs many processes at the same time (called multitasking).**

**The CPU is a limited resource, and all processes cannot use it at once.**

- Some processes take more time, while others are short.
- Without scheduling, some processes might wait too long or never get a chance.

**So, to maintain fairness and efficiency, scheduling is necessary.**

## Types of Scheduling

There are different types of scheduling:

**Long-Term Scheduling – decides which processes are admitted into the system.**

**Short-Term Scheduling (CPU scheduling) – decides which ready process runs next.**

**Medium-Term Scheduling – temporarily removes processes to free up memory.**

**But when we say CPU Scheduling, we usually refer to short-term scheduling.**

## Terminologies Used in CPU Scheduling

**Arrival Time: The time at which the process arrives in the ready queue.**

**Completion Time: The time at which the process completes its execution.**

**Burst Time: Time required by a process for CPU execution.**

**Turn Around Time: Time Difference between completion time and arrival time.**

Turn Around Time = Completion Time – Arrival Time

**Waiting Time(W.T): Time Difference between turn around time and burst time.**
Waiting Time = Turn Around Time – Burst Time

---

## CPU and I/O burst cycle:

Every process alternates between two types of activities:

### CPU Burst

- Time when the process is using the CPU to execute instructions.
- Example: Calculating, processing logic, etc.

### I/O Burst

- Time when the process is waiting for input or output (I/O) operations.
- Example: Reading from a file, writing to disk, waiting for user input.

**A process does not keep using the CPU continuously. Instead, it uses the CPU for a while (CPU burst) and then waits for I/O (I/O burst). This cycle repeats until the process finishes.**

---

## Alternating Sequence of CPU And I/O Bursts

## Fig: Alternating Sequence of CPU And I/O Bursts.

### Why Burst Cycle is Important in Scheduling

Understanding the CPU and I/O burst cycle helps in:

- Choosing the best scheduling algorithm.
- Balancing CPU and I/O usage.
- Ensuring smooth and efficient execution of all processes.

If the scheduler selects a process wisely (like choosing a short CPU burst process first), it can reduce waiting time for others and improve system performance.

---

## Preemptive and Non-Preemptive Scheduling

**In operating systems, CPU scheduling is used to decide which process should run on the CPU next. This can be done in two ways:**

### ■ 1. Preemptive Scheduling

**The CPU can be taken away from a running process if needed.**

**This is useful when a higher priority process arrives.**

**It is more complex but gives better responsiveness.**

**Allows fair use of the CPU among all processes.**

- Preemptive scheduling is used when a process switches from running state to ready state or from the waiting state to the ready state.

### Example:

Think of a traffic signal. If an ambulance comes, it is given priority even if other vehicles are waiting.

Similarly, in preemptive scheduling, high-priority or short tasks can interrupt running processes.

### Examples of Preemptive Algorithms:

- Round Robin (RR)
- Shortest Remaining Time First (SRTF)
- Priority Scheduling (Preemptive)

College Sahayak

### ■ 2. Non-Preemptive Scheduling

• Once a process gets the CPU, it keeps it until it finishes or goes to waiting (I/O).

**The CPU cannot be taken away from the process.**

**It is simpler and easier to implement.**

**But it can be unfair to short processes if a long process is already running.**

• Non-Preemptive scheduling is used when a process terminates , or when a process switches from running state to waiting state.

**Example:**

Imagine a single-lane road. If a car enters, other cars must wait until it finishes its journey. Similarly, in non-preemptive scheduling, once a process starts, others have to wait.

**Examples of Non-Preemptive Algorithms:**

• First-Come, First-Served (FCFS)
• Shortest Job First (SJF – Non-preemptive)

---

## Scheduling Criteria

**The scheduling criteria for CPU scheduling algorithms include the following key metrics:**

### ■1. CPU Utilization

• Shows how busy the CPU is.

**Goal: Keep CPU as busy as possible (ideally 100%).**

### ■ 2. Throughput

• Number of processes completed in a given time.
• Higher throughput = better performance.

### ■ 3. Turnaround Time

**Time taken from process submission to completion.**

• Formula:

Turnaround Time = Completion Time - Arrival Time

### ■ 4. Waiting Time

**Time a process spends in the ready queue, waiting for the CPU.**

• Lower waiting time = better scheduling.

### ■ 5. Response Time

**Time between submitting the process and the first response from CPU.**

• Important in real-time systems where quick response is needed.

---

## Types of Scheduling Algorithms:

**The CPU scheduling algorithm is a method used by the operating system to choose which process will run next on the CPU. Below are some commonly used scheduling algorithms:**

---

## First-Come First-Serve Scheduling, FCFS:

**It is the simplest scheduling algorithm.**

**-The process that comes first is served first.**

College Sahayak

**It is non-preemptive, meaning once a process starts, it runs until it finishes.**

- Like people standing in a line at a ticket counter – the person who comes first gets the ticket first.

-In this algorithm, a process, that a request the CPU first, is allocated the CPU first.

-FCFS scheduling is implemented with a FIFO queue.

-When the CPU is available, it is allocated to the process at the head of the queue.

-Once the CPU is allocated to a process, that process is removed from the queue.

-The process releases the CPU by its own.

-Easy to understand and implement.

-Its implementation is based on FIFO queue.

-Poor in performance as average wait time is high.

---

## Example 01:

Suppose that the processes arrive in the order: P1, P2, P3 The Gantt Chart for the schedule is:

- Consider order: P1→P2→P3:
- Gantt chart:

**Average waiting time = (0 + 24 + 27) / 3 = 17 ms.**

---

## Advantages:

- Easy to implement
- Fair for processes that come first

---

## Disadvantages:

**Long waiting time for short processes if a long one comes first**

**Convoy effect – long process delays all others**

---

## Shortest-Job-First Scheduling, SJF:

- This is a non-preemptive, pre-emptive scheduling algorithm.
- Best approach to minimize waiting time.
- Technically this algorithm picks a process based on the next shortest CPU burst, not the overall process time.
- Easy to implement in Batch systems where required CPU time is known in advance.
- Impossible to implement in interactive systems where required CPU time is not known.
- The processer should know in advance how much time process will take.

**The process with the shortest CPU burst time is selected next.**

---

## Example 1:

### Given processes:

- By SJF scheduling:
- Gantt chart:
- Average waiting time = (3 + 16 + 9 + 0) / 4 = 7 ms.

---

## Example 2:

- Given processes:

College Sahayak

- By preemptive SJF scheduling:
- Gantt chart:
- Average waiting time = [(10 -1) +(1 - 1) +(17 - 2) + (5 - 3)] /4 = 26 /4 = 6.5 ms.

## Advantages:

- Reduces average waiting time

## Disadvantages:

- Hard to know the burst time in advance

**May lead to starvation for longer processes.**

## Shortest Remaining Time Next (SRTN):

**It is the preemptive version of SJF.**

- the pre-emptive version of Shortest Job First (SJF) scheduling, called Shortest Remaining Time First (SRTF).

**CPU is given to the process with the shortest remaining time.**

- If a new process arrives with a shorter time, the current process is preempted.

## Example 1: Processes with Same Arrival Time

Consider the following table of arrival time and burst time for three processes P1, P2 and P3.

### Step-by-Step Execution:

- Time 0-5 (P3): P3 runs for 5 ms (total time left: 0 ms) as it has shortest remaining time left.
- Time 5-11 (P1): P1 runs for 6 ms (total time left: 0 ms) as it has shortest remaining time left.
- Time 11-19 (P2): P2 runs for 8 ms (total time left: 0 ms) as it has shortest remaining time left.

### Gantt chart:

As we know,

**Turn Around time = Completion time - arrival time**

**Waiting Time = Turnaround time - burst time**

Now,

**Average Turnaround time = (11 + 19 + 5)/3 = 11.6 ms**

**Average waiting time = (5 + 0 + 11 )/3 = 16/3 = 5.33 ms**

## Example 2: Processes with Different Arrival Times

Consider the following table of arrival time and burst time for three processes P1, P2 and P3.

### Step-by-Step Execution:

- Time 0-1 (P1): P1 runs for 1 ms (total time left: 5 ms) as it has shortest remaining time left.
- Time 1-4 (P2): P2 runs for 3 ms (total time left: 0 ms) as it has shortest remaining time left among P1 and P2.
- Time 4-9 (P1): P1 runs for 5 ms (total time left: 0 ms) as it has shortest remaining time left among P1 and P3.
- Time 9-16 (P3): P3 runs for 7 ms (total time left: 0 ms) as it has shortest remaining time left.

### Gantt chart:

Now, let's calculate average waiting time and turnaround time:

- Average Turn around time = (9 + 14 + 3)/3 = 8.6 ms

College Sahayak

• Average waiting time = (3 + 0 + 7 )/3 = 10/3 = 3.33 ms

## Advantages:

• Better average turnaround and waiting time than non-preemptive SJF.

## Disadvantages:

**Can lead to starvation of long processes**

• Requires accurate prediction of remaining time

## Round Robin (RR):

• RR is similar to FCFS except that preemption is added to switch between processes.
• Round robin scheduling is similar to FCFS scheduling, except that CPU bursts are assigned with limits called time quantum.

**Each process is given a fixed time slice or quantum (e.g., 2 ms).**

• After the time ends, the process is moved to the back of the queue if not finished.

**It is preemptive.**

• Like giving every student in a class 2 minutes to speak. After 2 minutes, the next student gets the chance.
• Goal: fairness, time sharing.

## Example 1:

Scenario 1: Processes with Same Arrival Time

Consider the following table of arrival time and burst time for three processes P1, P2 and P3 and given Time Quantum = 2 ms

### Step-by-Step Execution:

**Time 0-2 (P1): P1 runs for 2 ms (total time left: 2 ms).**

**Time 2-4 (P2): P2 runs for 2 ms (total time left: 3 ms).**

**Time 4-6 (P3): P3 runs for 2 ms (total time left: 1 ms).**

**Time 6-8 (P1): P1 finishes its last 2 ms.**

**Time 8-10 (P2): P2 runs for another 2 ms (total time left: 1 ms).**

**Time 10-11 (P3): P3 finishes its last 1 ms.**

**Time 11-12 (P2): P2 finishes its last 1 ms.**

Now, lets calculate average waiting time and turn around time:

• Turnaround Time = Completion Time - Arrival Time
• Waiting Time = Turnaround Time - Burst Time

**Average Turn around time = (8 + 12 + 11)/3 = 31/3 = 10.33 ms**

**Average waiting time = (4 + 7 + 8)/3 = 19/3 = 6.33 ms**

## Scenario 2: Processes with Different Arrival Times

Consider the following table of arrival time and burst time for three processes P1, P2 and P3 and given Time Quantum = 2

Step-by-Step Execution:

College Sahayak

### Time 0-2 (P1 Executes):

- P1 starts execution as it arrives at 0 ms.
- Runs for 2 ms; remaining burst time = 5 - 2 = 3 ms.

**Ready Queue: [P1].**

### Time 2-4 (P1 Executes Again):

- P1 continues execution since no other process has arrived yet.
- Runs for 2 ms; remaining burst time = 3 - 2 = 1 ms.
- P2 arrive at 4 ms.

**Ready Queue: [P2, P1].**

### Time 4-6 (P2 Executes):

- P2 starts execution as it arrives at 4 ms.
- Runs for 2 ms; remaining burst time = 2 - 2 = 0 ms.
- P3 arrive at 5ms

**Ready Queue: [P1, P3].**

### Time 6-7 (P1 Executes):

- P1 starts execution.
- Runs for 1 ms; remaining burst time = 1 - 1 = 0 ms.

**Ready Queue: [P3].**

### Time 7-9 (P3 Executes):

- P3 starts execution.
- Remaining burst time = 4 - 2 = 2 ms.

**Ready Queue: [P3].**

### Time 9-11 (P3 Executes Again):

- P3 resumes execution and runs for 2 ms and complete its execution
- Remaining burst time = 2 - 2 = 0 ms.

### Ready Queue: [].

**Average Turn around time =7+2+6/3=15/3=5ms**

**Average waiting time = 2+0+2/3=1.33ms**

---

## Advantages:

**Good for time-sharing systems**

- All processes get equal attention

---

## Disadvantages:

- Too short time quantum = too many context switches
- Too long time quantum = becomes like FCFS

---

## Priority Scheduling:

- Priority scheduling is a more general case of SJF, in which each job is assigned a priority and the job with the highest priority gets scheduled first. ( SJF uses the inverse of the next expected burst time as its priority - The smaller the expected burst, the higher the priority.)

College Sahayak

**Each process is assigned a priority number.**

**Process with highest priority gets the CPU first.**

**Can be preemptive or non-preemptive.**

- Each process is assigned a priority. Process with highest priority is to be executed first and so on.
- Processes with same priority are executed on first come first served basis.
- Priority can be decided based on memory requirements, time requirements or any other resource requirement.

## Example 1 :

- Given processes:
- By preemptive SJF scheduling:
- Gantt chart:
- Average waiting time = (6 + 0 + 16 + 18 + 1) / 5 = 8.2 ms.

## Advantages:

- Important tasks are completed faster

## Disadvantages:

**Low-priority processes may starve**

- Starvation can be solved using aging (gradually increasing priority of waiting processes)

## Multilevel Queue Scheduling:

- Multiple queues are maintained for processes with common characteristics.
- Each queue can have its own scheduling algorithms.
- Priorities are assigned to each queue.
- Processes are divided into groups (queues) based on their type (e.g., system, interactive, batch).

**Each queue has its own scheduling algorithm.**

- There is a rule to select which queue's process gets CPU next.

**Example Queues:**

- System processes → Priority Scheduling
- User processes → Round Robin

## Advantages:

- Organizes different types of processes well
- Flexible and can be tuned for better performance

## Disadvantages:

- Hard to manage and balance between queues
- Risk of starvation for lower-priority queues

## Deadlock:

- In an operating system, deadlock is a situation where a group of processes are waiting for each other forever and none of them can continue. All the processes are blocked and the system cannot proceed further.
- Imagine four cars stuck at a four-way signal, each waiting for the other to move first — no one moves. That's what happens in a deadlock.

College Sahayak (c) 2026 | Verified MSBTE Study Material | All Rights Reserved

- In the diagram below, for example, Process 1 is holding Resource 1 while Process 2 acquires Resource 2, and Process 2 is waiting for Resource 1.

**Fig: Deadlock in Operating system**

---

## System Model:

- In an operating system, many processes run at the same time. These processes need

**resources like CPU, memory, files, printers, etc., to complete their tasks.**

- The System Model explains how processes and resources interact, and how deadlock can happen when this interaction goes wrong.

---

## Types of Resources

There are two types of system resources:

### Preemptable Resources

**These resources can be taken away from a process without causing problems.**

- Example: CPU, memory.

### Non-preemptable Resources

**These resources cannot be taken back once given to a process.**

**The process must release them voluntarily.**

- Example: Printers, files.

**Deadlocks usually happen with non-preemptable resources, because they can't be forcefully taken back.**

---

## Process and Resource Behavior

A process follows these steps to use a resource:

**Request the resource**

**Use the resource**

**Release the resource**

**If a resource is not available, the process waits (gets blocked).**

**Deadlock occurs when processes hold some resources and keep waiting for others, which are held by other blocked processes — forming a cycle of waiting.**

---

## Necessary Conditions for Deadlock:

**There are four conditions that must happen together for a deadlock to occur. If even one of these is missing, deadlock won't happen.**

### Mutual Exclusion

- Only one process can use a resource at a time.

**If another process wants the same resource, it has to wait.**

### Example:

Only one process can use the printer at a time.

### Hold and Wait

College Sahayak

**A process is holding at least one resource and waiting for more.**

- It does not release the resources it already has.

### Example:

Process A is using the printer and waiting for a file. Process B is using the file and waiting for the printer.

→ **Both are holding and waiting.**

### No Preemption

**A resource cannot be taken forcibly from a process.**

**It must be released voluntarily by the process after finishing.**

### Example:

If Process A has the printer, it will not give it up until it's done — even if Process B needs it urgently.

### Circular Wait

**There is a cycle of processes, each waiting for a resource held by the next.**

### Example:

- Process A waits for a file held by B
- B waits for a printer held by C
- C waits for a memory block held by A

→ **They are stuck in a circular wait.**

---

## Deadlock Handling:

- Deadlock occurs when two or more processes are stuck, each waiting for a resource that another process is holding. To solve this, operating systems use deadlock handling methods.

**There are three main ways to handle deadlocks:**

**Deadlock Prevention**

**Deadlock Avoidance**

**Deadlock Detection and Recovery**

### In this explanation, we'll focus on the first two.

Deadlock Prevention:

- The Concept of deadlock prevention is like a normal, the very important phrase is "Prevention is better than cure".
- The Method is the time when the deadlock occurs, we are trying to find the solution better is that before the deadlock occurs we try to find some solutions.
- Deadlock prevention means we change the system rules so that at least one of the four deadlock conditions never occurs.

Let's recall the 4 conditions for deadlock:

**Mutual Exclusion**

**Hold and Wait**

**No Preemption**

**Circular Wait**

### Now let's see how we can prevent deadlock by breaking these:

### ■ A. Mutual Exclusion

College Sahayak

- Not always preventable.
- Some resources must be used one at a time (like printers), so mutual exclusion is necessary.

### ■ B. Hold and Wait

- We can stop hold and wait by forcing each process to request all required resources at once, before starting.

**But this can cause low resource utilization and long waiting time.**

### ■ C. No Preemption

- Allow preemption: If a process is waiting and cannot get a resource, then take back the resources it already has and give them to others.

**This method works, but not all resources can be preempted.**

### ■ D. Circular Wait

- Prevent circular wait by assigning a fixed order to all resource types (like R1 < R2 < R3).

**A process can only request resources in increasing order, not randomly.**

- This prevents any cycle from forming.

---

## Deadlock Avoidance:

- Instead of strict prevention, deadlock avoidance means the system checks before allocating resources whether the allocation is safe or not.
- Means when we give the resources to a particular, we check whether deadlock can occur or not? Or deadlock can be avoided or not?

### This method needs the system to know in advance:

- How many resources each process might need
- Which resources are currently free
- What is already allocated
- For every resources allocation ,we check safe and Unsafe Situation.
- This is done by Banker’s Algorithm.

**The Dijkstra’s Algorithm is also known as the Banker’s Algorithm.**

---

## Banker’s Algorithm (By Dijkstra):

- This is the most popular deadlock avoidance algorithm, used in systems like banks
- We also called Banker’s algorithm as Deadlock Avoidance Algorithm.
- Just like a bank only gives loans if it knows it has enough funds to meet future needs, the system only gives resources if it will still be in a safe state.
- Banker's algorithm is a deadlock avoidance algorithm. It is named so because this algorithm is used in banking systems to determine whether a loan can be granted or not.

### Characteristics of Banker's Algorithm

The characteristics of Banker's algorithm are as follows:

- If any process requests for a resource, then it has to wait.
- This algorithm consists of advanced features for maximum resource allocation.
- There are limited resources in the system we have.
- In this algorithm, if any process gets all the needed resources, then it is that it should return the resources in a restricted period.
- Various resources are maintained in this algorithm that can fulfill the needs of at least one client.

### Important Terms in Banker’s Algorithm:

College Sahayak

**Available – Resources currently free.**

**Max – Maximum resources each process may need.**

**Allocation – Resources currently given to a process.**

**Need = Max – Allocation**

---

## Steps of Banker's Algorithm:

**Check if Need ≤ Available for any process.**

**If true, temporarily allocate resources to that process.**

- Update the Available list.
- Repeat for all processes.

**If all processes can finish in some order → safe state.**

**If not → don't give the resource → unsafe state (avoid allocation).**

---

## Example of Banker's Algorithm:

College Sahayak

# UNIT IV Memory Management

*CO4: Analyze Memory Management techniques used by an OS. Total Marks: 14*

**Syllabus: Total Marks:14**

## basic memory management:

- Memory management is the functionality of an operating system which handles or manages primary memory.
- Memory management keeps track of each and every memory location either it is allocated to some process or it is free.
- It checks how much memory is to be allocated to processes. It decides which process will get memory at what time.
- It tracks whenever some memory gets free do run allocated and correspondingly it updates the status.
- Two important terms considered while loading programs into the partition are internal fragmentation and external fragmentation.
- Memory Management means how the operating system manages the computer's RAM (main memory).

### Partitioning:

- Partitioning is the process of dividing memory into smaller parts (called partitions) to load programs.

There are two main types:

### ■1. Fixed Partitioning :(Static partition)

- The memory is divided into fixed-size blocks before running any program.
- Each block can hold only one process.
- If a process is smaller than the block, the extra space is wasted.\
- Memory is divided into fixed-sized partitions, and each partition can hold only one process.
- The OS keeps track of free and occupied partitions using a partition table.

### Example:

**If a block is 100 KB and the process needs 60 KB, 40 KB is wasted. This is called internal fragmentation.**

■ Example:

Suppose RAM = 400 KB is divided into 4 fixed partitions of 100 KB each.

Now, if a program of 70 KB is loaded into one partition, then 30 KB remains unused.

Disadvantages:

- Internal fragmentation (unused memory within partition).
- Limited number of processes depending on number of partitions.
- Cannot adjust partition size dynamically.

## ■2. Variable Partitioning :

- The memory is divided dynamically, based on the size of the process.
- This saves space because each partition is made as per the process size.
- In variable partitioning, memory is not divided in advance. Instead, partitions are created dynamically when a process arrives, according to its size.

### Example:

College Sahayak

If a process needs 60 KB, only 60 KB of memory is allocated.

### ■ Characteristics:

• No fixed partition sizes.

**Each process gets exact memory it needs.**

**No internal fragmentation (less memory waste inside partition).**

**But can have external fragmentation (free memory scattered across RAM).**

### ■ Example:

If a program of 70 KB arrives, the OS will create a 70 KB block for it in RAM. If another of 120 KB comes, another 120 KB block is created.

### Disadvantages:

• Memory becomes scattered over time (external fragmentation).
• Requires more complex memory management.

---

## Free Space Management Techniques:

**Free space management is an important job of the operating system.**

**It means keeping track of empty space on the hard disk or storage device.**

• The operating system must manage this space properly so files can be saved and used easily.
• To do this, the OS uses different methods.
• These methods help the system use storage space in the best way.
• Here are some common techniques used to manage free space:

**Bit Map (Bit Vector)**

**Linked List**

**Grouping**

**Counting**

**When a file is saved or a program runs, it needs free memory space.**

• The operating system (OS) uses Free Space Management Techniques to keep track of

**which parts of memory are free and which are used.**

---

## Bit Map (or Bit Vector) :

**- A Bitmap is a list of bits (0s and 1s).**

**-Each bit represents one block of the disk.**

**-It is also called a Bit Vector.**

**What do 0 and 1 mean?**

**0 = The block is free (empty).**

**1 = The block is allocated (used).**

**How does it work?**

**-Suppose the disk has 16 blocks.**

-Each block has one bit in the bitmap.

-So, we will have 16 bits in the bitmap.

College Sahayak

### Example:

Let's say block numbers 1 to 4, 8 to 12, and 15 are used (allocated), then the bitmap looks like:

- Green (used) blocks = 1
- White (free) blocks = 0

So, blocks 5, 6, 7, 14, 15 are free.

### Fig : Bit map

### Advantages of Bitmap:

- Easy to understand and use.

**Good for small-sized disks.**

- Can quickly tell which blocks are free or used.

### Disadvantages of Bitmap:

**Bitmap becomes large if the disk has many blocks.**

**Takes more memory space for large storage.**

**Finding a continuous group of free blocks may take more time.**

---

## Linked List :

**In this method, all free blocks on the disk are connected like a chain.**

**Each free block has a pointer (address) to the next free block.**

- The first free block's number is saved separately on the disk and also in RAM (cached).
- The last free block has a NULL pointer, which means there are no more free blocks after it.

---

## How does it work? (Example)

**Look at the Figure - 2 (provided):**

**■ This figure shows a disk with blocks from Block1 to Block16.**

**Green blocks are already used (allocated).**

**White blocks are free.**

**All white (free) blocks are connected using pointers.**

**■ The Linked List is like this:**

**Free list head → points to Block5**

**Block5 → points to Block6**

**Block6 → points to Block9**

**Block9 → points to Block14**

**Block14 → points to Block15**

**Block15 → points to NULL**

**■ This chain is called the Free Space List.**

---

## Advantages of Linked List Method:

**Efficient use of space – All free blocks are used properly.**

College Sahayak

**Dynamic allocation – You can add or remove blocks easily as per need.**

**No large memory table is needed like in the bitmap method.**

## Disadvantages of Linked List Method:

• If the number of free blocks is large, then managing pointers becomes difficult.

**To find a specific free block, you must go through the list step-by-step.**

**It needs more I/O operations to access all the free blocks from disk.**

## Swapping:

**-Swapping is the process of moving programs or data between RAM (main memory) and secondary memory (like a hard disk or SSD).**

-It helps the system when RAM is full and more programs need to run.

-Swapping allows more programs to run at the same time, even if RAM is small.

-It is only used when the data needed is not already in RAM.

-Swapping can slow down the system, but it helps run big programs.

**-That's why it is sometimes called memory compaction.**

## How it works:

The CPU scheduler decides which program should go out of RAM (swap out) and which one should come in (swap in).

Example: If a high-priority program comes, a low-priority one is moved out. When the high-priority task finishes, the low-priority one comes back in.

## Two main steps of Swapping:

**Swap-Out: Moves a process from RAM to hard disk to free space.**

**Swap-In: Brings a process from hard disk to RAM when needed.**

## Swapping Process: Step-by-Step

• RAM is full.
• OS chooses a process that is not being used right now.
• That process is moved from RAM to the hard disk (swap-out).
• This gives space for a new process.
• When the old process is needed again, it is brought back into RAM (swap-in).
• Another unused process may be moved out to make space.

## Real-Life Example:

**Think of your RAM as a small study table, and your hard disk as a cupboard. You keep only important books on your table.**

If you need a new book from the cupboard, you put away a book from the table. This way, you manage space and always work with the needed books.

College Sahayak

## Advantages of Swapping:

- It reduces waiting time of processes by making extra space using swap.
- It frees up RAM by moving unused data to hard disk.
- This makes RAM available for active programs.
- Even if RAM is small, swapping helps run large programs.
- It avoids overload by removing inactive programs from RAM.
- This keeps active processes running smoothly.

## Disadvantages of Swapping:

- If power is lost while swapping, data can be lost.

**Swapping causes more page faults, which slows the system.**

- If too many programs keep swapping, performance drops.
- CPU wastes time in swapping instead of running programs.

## Compaction:

**-Compaction is used in memory management to solve the problem of external fragmentation.**

**-It means moving all free memory spaces together into one big block.**

**-This helps in storing big processes that need more memory.**

**-It is also called Defragmentation.**

- Compaction is the process of moving programs together in memory to make one big continuous free space.

**It removes the gaps between memory blocks.**

## Example:

If memory has free spaces like this:

after compaction it becomes:

## Advantages:

**Gives larger continuous memory space.**

**Reduces external fragmentation.**

## Disadvantages:

**Takes extra time (data movement).**

- Cannot be used if memory blocks are fixed.

## Fragmentation:

- Fragmentation means memory is wasted because it's not used properly. There are two types:

## ■ A. Internal Fragmentation

**Happens when memory is divided into fixed blocks.**

**If a process does not fully use the block, the remaining space is wasted.**

**Happens inside memory blocks.**

College Sahayak

• Example: You get a 64KB block but use only 40KB → 24KB is wasted inside the block.

**Example:**
Block size = 100 KB Process size = 70 KB

Wasted = 30 KB (this is internal fragmentation)

---

## ■ B. External Fragmentation

**Happens when memory is free, but not together.**

**So, even if free memory is enough, it can't be used continuously.**

**Happens outside the memory blocks.**

**Example: Free space is there but in small pieces, not together.**

**So, system can't fit big processes even if total space is enough.**

---

## Example:

Free blocks: 40 KB +10 KB + 5 KB Needed = 50 KB

Total free = 55 KB but not in one place → process can't be loaded.

---

## Partitioning Algorithms:

**Partitioning means dividing memory into smaller parts.**

**It helps run many programs at once without mixing them.**

• Like dividing a big room into smaller rooms for different people.

**Helps OS organize memory and use it efficiently.**

• Partitioning Algorithms are used to allocate free memory blocks to new processes.

**There are different Placement Algorithm:**

**First Fit**

**Best Fit**

**Worst Fit**

---

## First Fit

**It selects the first free block that is big enough for the process.**

**Finds the first memory block that fits the process.**

• Stops scanning once it finds a suitable block.

### ■ Example:

**Given:**
We have a process:

**Process A = 25 KB**
And the memory blocks are:

### ■ Step-by-Step (First Fit):

**Start checking memory from the top.**

College Sahayak

**Block 1 → 20 KB → Too small ■**

**Block 2 → 15 KB → Too small ■**

**Block 3 → 40 KB → Free and enough ■**

➡ So, Process A (25 KB) is placed in Block 3.

### ■ Result:

**Used: 25 KB of 40 KB**

**Remaining Hole: 40 - 25 = 15 KB**

**This unused 15 KB space is called Internal Fragmentation, because it's left inside the allocated block.**

### ■ Final Output:

**Process A placed in Block 3**

**Internal Fragmentation = 15 KB**

- First Fit works by choosing the first available space that fits the process size.

---

## Advantage:

-Simple and easy to use.

- Fast to allocate.
- Uses less CPU.

---

## Disadvantage:

- Can create small unused spaces (fragmentation).
- May waste memory.
- Can get slower over time.

---

## Best Fit

**It selects the smallest block that is enough for the process.**

**Finds the smallest block that fits the process.**

- Good for saving memory.
- The Best Fit algorithm finds the smallest hole (empty space) that is big enough to fit the process.

**This helps in reducing wastage of memory (i.e., small leftover holes).**

### ■ Example:

Process A = 25 KB

### How Best Fit Works Here:

**We want to insert Process A (25 KB).**

- Best Fit will look for a hole (free space) that is just big enough or exactly matching.
- In this example:

### o There is only one hole available that is 25 KB in size.

**Since Process A = 25 KB, it fits perfectly into the 25 KB hole.**

**There is no remaining space left (hole = 25 - 25 = 0 KB), which is ideal.**

---

## Advantage:

College Sahayak

-Reduces unused memory.

- Better space usage.

-Leaves large blocks free for other processes.

---

## Disadvantage:

-Slow: has to scan all blocks.

- Creates very small gaps.
- Not always "best" in real use.

---

## Worst Fit

- The Worst Fit algorithm always selects the largest available hole in memory to fit the process.

**This may leave larger leftover holes, causing fragmentation.**

**It selects the largest available block.**

**Chooses the largest memory block for a process.**

- Leaves large leftover space.

### ■ How Worst Fit Works Here:

- We need to allocate Process A (25 KB).
- Worst Fit algorithm searches for the largest hole in memory.
- The largest available hole is in Block 4 = 60 KB.
- So, Process A is placed in Block 4.
- After placing 25 KB, the remaining hole is: 60 KB - 25 KB = 35 KB

### ■ Conclusion (Worst Fit):

- Process A is allocated in the largest available hole.
- Memory left unused = 35 KB, which is wasted.
- This can cause external fragmentation (free memory, but not usable).

---

## Advantage:

- Large processes can be handled easily.

-Leaves bigger gaps that can be reused.

-Leaves bigger remaining block

---

## Disadvantage:

- Wastes space (over-allocates).
- Slow due to full scan.
- Not efficient in long run.
- Wastes more space

**In this method, a process is not stored in one single block of memory.**

- Instead, the process is divided into smaller parts, and each part is stored in different free spaces in main memory.

**This helps when enough continuous (contiguous) memory is not available.**

- In noncontiguous memory allocation, it is allowed to store the processes in non contiguous memory locations. There are different techniques used to load processes into memory, as follows:

**1. Paging**

College Sahayak

**2. Segmentation**

**3. Virtual memory paging(Demand paging) etc.**

---

## PAGING:

**Process is divided into equal parts called Pages.**

**Memory is divided into equal parts called Frames.**

**Each Page fits into a Frame.**

**Pages can be placed anywhere in memory.**

**No external fragmentation occurs.**

**Page Table is used to keep track of page locations.**

- Main memory is divided into a number of equal-size blocks, are called frames.
- Each process is divided into a number of equal-size block of the same length as frames, are called Pages.
- A process is loaded by loading all of its pages into available frames (may not be contiguous).

**Fig:Diagram of Paging hardware.**

---

## Process of Translation from logical to physical addresses

- Every address generated by the CPU is divided into two parts: a page number (p) and a page offset (d).
- The page number is used as an index into a page table.
- The page table contains the base address of each page in physical memory.
- This base address is combined with the page offset to define the physical memory address that is sent to the memory unit.
- If the size of logical-address space is 2m and a page size is 2n addressing units (bytes or words), then the high-order (m–n)bits of a logical address designate the page number and the n low-order bits designate the page offset. Thus, the logical address is as follows:

- Where p is an index into the page table and d is the displacement within the page

---

## Advantages of Paging :

### No External Fragmentation

o Memory is divided into fixed-size blocks, so there’s no waste between memory areas.

### Better Memory Use

- Even small empty spaces can be used to store parts of a program.

### Supports Virtual Memory

- Programs can run even if they are bigger than the actual RAM.

### Easy Swapping

- Only small parts (pages) move in/out of memory, not the whole program.

### Better Security

- Each program stays in its own memory space and can’t touch others.

---

## Disadvantages of Paging:

### Internal Fragmentation

- Last page may have unused space, which is wasted.

College Sahayak

### More Memory Use
• Page tables need extra memory to store address info.

### Slower Access
• Finding the real memory address takes extra steps.

### Page Fault Delay
• If a page is not in memory, it must be loaded from disk, which is slow.

### More Complex System
• Needs special hardware and software (like MMU, TLB, etc.) to manage paging.

---

## Segmentation:

**Process is divided into logical parts called Segments (like code, stack, data).**

**Each segment is stored in different memory locations.**

**Each segment can be of different size.**

**Segment Table keeps record of each segment's location.**

**May cause external fragmentation.**
• Segmentation is a memory-management scheme that supports user view of memory.
• A program is a collection of segments.

**A process is divided into parts called segments (like code, data, stack).**

**Segments are not fixed in size.**

**Segmentation shows how a user sees the memory (unlike paging).**

**These segments are stored in physical memory using a Segment Table.**

---

## Types of Segmentation

**Virtual Memory Segmentation: Segments are created as needed, not all at once.**
• Simple Segmentation: All segments are created and loaded when the process runs (not always next to each other).

---

## What is Segment Table?

It maps a two-dimensional Logical address into a one-dimensional Physical address. It's each table entry has:
• Base Address: It contains the starting physical address where the segments reside in memory.
• Segment Limit: Also known as segment offset. It specifies the length of the segment.

---

## Fig: Diagram of Segmentation Hardware

### Logical to Physical Address
• CPU makes a logical address with:

**Segment Number (s): Tells which segment.**

**Offset (d): Position inside that segment.**
• This is converted to a physical address using the segment table.

---

## Advantages of Segmentation:

- Less internal fragmentation: Segment size is flexible as per need.
- Segment Table uses less memory than paging's page table.
- CPU usage improves as full segments are loaded.
- Easy to understand for users – similar to dividing code into modules.
- User decides segment size, not hardware.
- Can separate code and data for better security.
- Flexible: Segments can be of any size.
- Can share segments (like libraries) between processes.
- More secure: One process can't access another's memory.

---

## Disadvantages of Segmentation:

- External fragmentation: Free memory becomes scattered.
- Segment table needs memory and managing it adds extra work.
- Slower access: Needs two memory lookups (segment table + actual memory).
- More fragmentation: Wasted memory due to scattered segments.
- Overhead: More data and time to manage segments.
- Complex system: Harder to design and manage than paging.

---

## Virtual Memory:

**Virtual memory lets programs run even if the whole program is not in RAM.**

- Only the needed part of the program is loaded in memory.
- It means the logical memory (used by programs) can be bigger than the actual RAM.

**Virtual memory helps different programs share files and memory easily.**

**It also helps in creating new processes quickly.**

- It separates user memory and physical RAM, so the user thinks more memory is available.
- Programmers don't need to worry about how much real memory (RAM) is present.

**It makes it easy to run big programs on small RAM systems.**

---

## Objectives of Virtual Memory:

**Run many programs at once (multiprogramming support).**

**A program doesn't have to be fully in memory to run—just the part in use.**

**Programs can be bigger than the system's RAM.**

**It creates a false view of a big memory, even if RAM is small.**

**It uses both RAM and disk space to manage memory better.**

**Helps the system run more programs at the same time and use memory efficiently**

**Fig: Diagram showing virtual memory that is larger than physical memory.**

**Virtual memory can be implemented via:**

**Demand paging**

**Demand segmentation**

**Types of Virtual Memory :**

**How Virtual Memory Works**

**Virtual memory is handled by a Memory Management Unit (MMU).**

• The CPU gives virtual addresses, and MMU changes them into physical addresses (RAM).

## Paging

### ■ What is Paging?

**Paging splits memory into small equal parts called pages.**

**Pages not needed right now are stored on the hard disk (swap file).**

**When needed, pages are brought back to RAM — this is called page swapping.**

### ■ Demand Paging (When pages load only when needed) Steps:

**If a page is missing in memory, CPU gives an interrupt (called a page fault).**

**The OS pauses the process and looks for the missing page.**

**It finds the page in storage (logical space).**

**The page is brought from storage to RAM using page replacement algorithms.**

**Page table is updated to show the new page.**

**The OS tells the CPU to resume the process.**

### ■ Page Fault Service Time

**It means total time taken to fix a page fault (i.e., load the missing page).**

**Formula:**

Effective Memory Access Time = (p × s) + (1 – p) × m Where:

p = page fault rate

s = time to handle a page fault m = memory access time

### ■ Page vs Frame

**A page = part of virtual memory.**

**A frame = slot in physical RAM where a page fits.**

• Pages from a program are placed into frames so the program can run even if it’s bigger than RAM.

## Segmentation

### ■ What is Segmentation?

**Segmentation breaks memory into variable-size blocks called segments.**

**Each segment may store a function, array, or data block.**

**Unused segments can be moved to hard disk to save space.**

**A segment table keeps details of each segment like:**

• Is it in memory?
• Has it changed?
• Where is it stored?

**Segments are loaded only when needed.**

Page Replacement:

• When memory is full and a new page is needed, the OS must decide which page to remove.

**We use page replacement algorithms to do this.**

College Sahayak

**We test them using a reference string (a list of memory page requests).**

**When a program needs a page that is not in memory (page fault), and no free memory frame is available, the system does page replacement.**

### Steps of Page Replacement:

**Find the required page on the disk.**

### Check for a free frame:

**If free frame is available → Load the page into it.**

**If no free frame → Use Page Replacement Algorithm to:**

**Choose a victim page (a page to remove).**

- Save the victim page to disk.
- Update the page table to show the page is removed.

**Load the required page into the freed frame.**

### Update the page and frame tables.

**Restart the program from where it stopped.**

- If no frame is free, two operations happen:

**One page goes out to disk.**

**One page comes in from disk.**

### ■ This makes the process slower because of double work.

### Fig: Diagram of Page replacement.

Page Replacement Algorithms :

### Common Page Replacement Algorithm:

- First In First Out (FIFO)
- Least Recently Used (LRU)
- Optimal Page replacement
- Most Recently Used (MRU)

---

## First In First Out (FIFO):

**FIFO means "first come, first go" — like a queue at a shop.**

**The page that came into memory first will be removed first when memory is full.**

**Removes the oldest page (which came first).**

**Uses a queue: first page in → first page out.**

- Easy to implement.
- The simplest page-replacement Algorithm is FIFO.
- FIFO means ,the First item that came in is the first to go out.

**Belady's Anomaly: More frames can increase page faults (unexpected!).**

- Note: For some page-replacement algorithms, the page fault rate may increase as the number of allocated frames increases. This most unexpected result is known as Belady's anomaly.

---

## Example 1:

College Sahayak

Consider page reference string 1, 3, 0, 3, 5, 6, 3 with 3-page frames. Find the number of page faults using FIFO Page Replacement Algorithm.

- Initially, all slots are empty, so when 1, 3, 0 came they are allocated to the empty slots ---> 3 Page Faults.

**When 3 comes, it is already in memory so ---> 0 Page Faults.**

- Then 5 comes, it is not available in memory, so it replaces the oldest page slot i.e 1. ---> 1 Page Fault.
- 6 comes, it is also not available in memory, so it replaces the oldest page slot i.e 3 ---> 1 Page Fault.

**Finally, when 3 come it is not available, so it replaces 0 -■ 1-page fault.**

---

## Least Recently Used (LRU):

- In this algorithm, page will be replaced which is least recently used.

**Removes the page that hasn't been used for the longest time in the past.**

**Assumes past = future usage.**

**Needs extra effort to keep track of usage.**

- Replaces the page that has not been used for the longest time.
- More practical and efficient compared to FIFO.

### Two LRU Implementations:

### ■ a. Counter Method:

- Every time a page is used, note the current time (using a counter).

**Remove the page with the oldest time.**

### ■ b. Stack Method:

**Use a stack (or linked list).**

**Every time a page is used, move it to the top.**

**Remove the page from the bottom (least recently used).**

---

## Example 1:

Consider the page reference string 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3 with 4-page frames. Find number of page faults using LRU Page Replacement Algorithm.

- Initially, all slots are empty, so when 7 0 1 2 are allocated to the empty slots ---> 4 Page faults.
- 0 is already there so ---> 0 Page fault. when 3 came it will take the place of 7 because it is least recently used ---> 1 Page fault.
- 0 is already in memory so ---> 0 Page fault.
- 4 will takes place of 1 ---> 1 Page Fault.
- Now for the further page reference string ---> 0 Page fault because they are already available in the memory.

---

## Optimal Page replacement:

- In this algorithm, pages are replaced which would not be used for the longest duration of time in the future.

**Replaces the page that will not be used for the longest time in the future.**

**Gives the best result (lowest page faults).**

- Has the lowest page fault rate of all Algorithm.

### It is Simply " Replace the page that will not be used for the Longest periods time."

**Not practical (we don't know the future).**

College Sahayak

- Used as a benchmark for comparison.
- Difficult to implement because it requires future knowledge used mainly for comparison Studies.

---

## Example 1 :

Consider the page references 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3 with 4-page frame. Find number of page fault using Optimal Page Replacement Algorithm.

College Sahayak

# UNIT V File Management & Disk Scheduling

*CO5: Apply techniques for effective File Management in an OS. Total Marks: 14*

**Syllabus: Total Marks:10**

## File Concept:

**A file is a collection of related data or records.**

**It is treated as a single unit by users and the system.**

**Every file has a unique name. Each file has a unique name to identify it.**

**Files can be created and deleted.**

**Files are used to store programs and data like text, images, audio, video, etc**

**Files can be created when we need to store information.**

**They can be deleted when no longer required.**

- A file acts like a container that stores useful information (text, images, code, etc.).

**Files are used to store programs and data.**

- Programs include software or apps.
- Data can include documents, images, audio, and more.
- A named collection of related information recorded on secondary storage(e.g., disks)

## File Attributes:

**Every file has some information (attributes) stored by the OS.**

**Name – The name of the file (like report.docx).**

**Identifier – A unique ID used by the OS to identify the file.**

**Type – Shows the type of file (like text, video, etc.).**

**Location – The address where the file is stored on disk.**

**Size – The current size of the file in bytes or blocks.**

**Protection – Who can read, write, or execute the file.**

- Time/Date/User Info – When the file was created, modified, or accessed, and by whom.

## File Operations:

Operating System provides several operations to manage files:

**Create – Make a new file.**

**Read – Open and read data from a file.**

**Write – Add or modify data in a file.**

**Reposition (Seek) – Move to a specific position in a file to read or write.**

**Delete – Remove a file from storage.**

College Sahayak

**Truncate – Erase the content of a file but keep the file and its name.**

---

## File Types:

- A common technique for implementing file types is to include the type as part of the file name. the name is split into two parts a name and an extension. Various file types are shown in the following table.

**Files have extensions that define their type. Some common types:**

---

## File Structure (How Files Are Organized):

Files are made up of smaller parts:

**Field – A small piece of data (e.g., name, date).**

**Record – Group of fields (e.g., one student's info).**

**File – Group of records.**

**Database – Group of related files with relationships.**

### Example:

A telephone directory = A file with records. Each record has 3 fields: Name, Address, Phone number.

---

## File System Structure:

**A file system organizes and manages files on storage.**

**It also controls file access, protection, and storage structure.**

- A file system helps the operating system store, find, and retrieve files easily and efficiently from storage devices like hard disks. To manage this, the file system is divided into different layers. Each layer has a specific job.

**Let’s understand each layer from top to bottom using the diagram:**

### Application Programs

- These are the programs you use (e.g., MS Word, Notepad).
- When they need to open, read, write, or save a file, they send a request to the file system.

### Logical File System

**This layer checks file permissions (who can open or write the file).**

**It manages the file name, file path, and directory structure.**

**If the application is not allowed to access the file, it shows an error.**

**It handles the metadata (info about the file like size, type, date).**

### File Organization Module

**This layer organizes files into blocks to store them on the hard disk.**

- It maps logical blocks (used by programs) to physical blocks (used by hardware).

**It also handles free space management (keeps track of empty areas on the disk).**

### Basic File System

- This layer receives instructions from the file organization module.

**It is responsible for sending commands to the hardware (via I/O control).**

**Think of it as a messenger between the software and hardware.**

### I/O Control (Input/Output Control)

College Sahayak

- This part contains device drivers – special codes that help control hardware like hard disks.
- It handles interrupts (signals sent by devices to the CPU when they need attention).

### Devices

- These are the physical hardware components like your hard disk, SSD, or USB drive.

**This is where the file is finally stored or retrieved from.**

---

## File Access Methods:

-Files store information like data, text, numbers, and so on. But to use this information, it must be read (accessed) and loaded into the computer's memory.

-A collection of bits bytes or lines stored on secondary storage devices such as a hard drive (magnetic disks) is called a file.

-In operating systems file access methods are simply ways to read information from the systems memory.

-There are various ways in which we can access the files from the memory like:

**Sequential Access**

**Direct/Relative Access, and**

---

## Sequential Access:

- Most of the operating systems access the file sequentially. In other words, we can say that most of the files need to be accessed sequentially by the operating system.

**This is the simplest method to access a file.**

- Data is read in order, one record after another, starting from the beginning of the file.
- You can't skip to a particular part directly. You must go through all previous data.
- In sequential access, the operating system reads the file word by word, one after another.

**A pointer is used to keep track of where we are in the file.**

### It starts at the beginning (base address) of the file.

- When the user wants to read the first word, the pointer gives that word and then

### moves to the next word.

**This process continues until the end of the file is reached.**

- Even though modern systems also support direct access (jumping to any part) and indexed access (like using a table of contents), sequential access is still the most commonly used.
- Why? Because most common files like:

### Text files Audio files Video files

### …are usually read from start to end, which makes sequential access the best fit.

Beginning Current Position Target Record End

0 45 75 100

### Fig 1: sequential access file

---

## Advantages of Sequential Access :

- Easy to implement and understand.
- Reads data in natural (A–Z) order.
- Best for files like text, audio, and video which are used from start to end.

---

## Disadvantages of Sequential Access :

• Slow if you want to reach a record far ahead.

**Adding new records may require shifting many existing records.**

---

## Direct Access:

• It is also called Random Access (Relative Access).
• this method allows the computer to jump directly to any part (record/block) of the file.
• You don't need to go through earlier records to reach the one you want.
• The Direct Access is mostly required in the case of database systems. In most of the cases, we need filtered information from the database.
• The sequential access can be very slow and inefficient in such cases.
• Suppose a file has many records.
• You can directly read:

**→ Block 14, then Block 50, then Write to Block 7.**

**No need to go in order. You can access any block instantly.**

**Fig : Direct access**

---

## Advantages of Direct/Relative Access :

• Files can be accessed instantly, reducing average access time.

**No need to go through previous blocks to reach the desired record.**

---

## Disadvantages of Direct/Relative Access :

• Hard to implement due to its complex mechanism.
• May cause security risks, so extra protection is needed.

The allocation methods define how the files are stored in the disk blocks. There are three main disk space or file allocation methods.

**Contiguous Allocation**

**Linked Allocation**

**Indexed Allocation**

The main idea behind these methods is to provide:

**Efficient disk space utilization.**

**Fast access to the file blocks.**

---

## 1. Contiguous Allocation:

• As the name suggests that we are allocating memory in Contiguous to the blocks of our file.
• Means, the storage we have ,we are placing the data in contiguous way in the storage.

**"Contiguous" means side-by-side or next to each other.**

• In this method, all parts of a file are stored together in nearby blocks of the hard disk.

In the above image on the left side, we have a memory diagram where we can see the blocks of memory. At first, we have a text file named file1.txt which is allocated using contiguous memory allocation, it starts with the memory block 0 and has a length of 4 so it takes the 4 contiguous blocks 0,1,2,3. Similarly, we have an image file and video file

College Sahayak

named sun.jpg and mov.mp4 respectively, which you can see in the directory that they are stored in the contiguous blocks. 5,6,7 and 9,10,11 respectively.

Here the directory has the entry of each file where it stores the address of the starting block and the required space in terms of the block of memory.

### Advantages

- Simple to implement
- Fast access (supports sequential and direct)
- Minimum disk head movement
- Low seek time

### Disadvantages

**You must know the file size in advance before storing.**

**File size cannot increase later (no space in between).**

**Can lead to fragmentation – free space gets broken up.**

---

## Linked File Allocation:

- The Linked file allocation overcomes the drawback of contiguous file allocation.
- In this scheme, each file is a linked list of disk blocks which need not be contiguous.

**File is stored in random blocks linked together.**

**Each block points to the next block in the file.**

**Each block has a pointer to next block.**

- Example: Blocks 5 → 12 → 7 → 9
- No need for continuous space.

**Slow access, only sequential reading.**

**Extra space needed for pointers.**

- Blocks are stored anywhere on the disk, but each block has a pointer to the next.

In the above image on the right, we have a memory diagram where we can see memory blocks. On the left side, we have a directory where we have the information like the address of the first memory block and the last memory block.

In this allocation, the starting block given is 0 and the ending block is 15, therefore the OS searches the empty blocks between 0 and 15 and stores the files in available blocks, but along with that it also stores the pointer to the next block in the present block. Hence it requires some extra space to store that link.

---

## Advantages and Disadvantages

### Advantages

- There is no external fragmentation.
- The directory entry just needs the address of starting block.
- The memory is not needed in contiguous form, it is more flexible than contiguous file allocation.

### Disadvantages

- It does not support random access or direct access.
- If pointers are affected so the disk blocks are also affected.
- Extra space is required for pointers in the block.

---

College Sahayak

## Indexed File Allocation:

- in this scheme, a special block known as the Index block contains the pointers to all the blocks occupied by a file.

**One block (index block) stores addresses of all file blocks.**

**One index block holds all addresses of file blocks.**

- File blocks can be anywhere on disk.
- The indexed file allocation is somewhat similar to linked file allocation as indexed file allocation also uses pointers but the difference is here all the pointers are put together into one location which is called index block.

**One block (index block) stores addresses of all file blocks.**

- Example: Index block → [20, 35, 18, 41]

**Fast direct access.**

- No fragmentation.

**Wastes space for small files (due to index block).**

---

## Advantages and Disadvantages

### Advantages

- It reduces the possibilities of external fragmentation.
- Rather than accessing sequentially it has direct access to the block.

### Disadvantages

- Here more pointer overhead is there.
- If we lose the index block we cannot access the complete file.
- It becomes heavy for the small files.
- It is possible that a single index block cannot keep all the pointers for some large files.

---

## File Directories Structure:

- When many files are stored in a system, it becomes hard to manage them.
- So, files are grouped and stored in different partitions. Each group is called a directory.

**A directory structure helps organize these files.**

---

## Single-Level Directory:

- it is the simplest directory structure.
- All files are contained in the same directory which is easy to support and understand
- All files are in one directory.
- Easy to understand and use.
- A single level directory has a significant limitation, however, when the number of files increases or when the system has more than one user. Since all the files are in the same directory, they must have a unique name. If two users call their dataset test, then the unique name rule violated.

**Fig : Single-level directory structure**

---

## Advantages:

- Since it is a single directory, so its implementation is very easy.
- If the files are smaller in size, searching will become faster.
- The operations like file creation, searching, deletion, updating is very easy in such a directory structure.

---

College Sahayak

### Disadvantages:

- There may chance of name collision because two files can not have the same name.
- Searching will become time taking if the directory is large.
- The same type of files cannot be grouped together.

---

### Two-Level Directory:

- the disadvantage of single level directory is the confusion of files names between different users

**The solution to this problem is to create a separate directory for each user.**

- In the two-level directory structure, each user has their own user files directory (UFD).
- The UFDs have similar structures, but each lists only the files of a single user. System's master file directory (MFD) is searched whenever a new user id is created.
- Each user gets their own directory.
- A master directory (MFD) points to each user directory (UFD).

**Fig : Two-Level directory structure.**

---

### Advantages:

- We can give full path like /User-name/directory-name.
- Different users can have same directory as well as file name.
- Searching of files become more easy due to path name and user-grouping.

---

### Disadvantages:

- A user is not allowed to share files with other users.
- Searching will become time taking if the directory is large.
- Two files of the same type cannot be grouped together in the same user.

---

### Tree- Structured Directory:

- the tree-structure allows user to create their own sub- directories and organize their files accordingly.
- The tree has a root directory.
- Every file in the systems has a unique path name.
- A path is the path from the root through all the sub- directories to a specified file.
- A directory contains a set of files and or sub-directories.
- Tree directory structure of operating system is most commonly used in our personal computers. User can create files and subdirectories too, which was a disadvantage in the previous directory structures.
- Like a tree: root → sub-directories → files.
- Users can create folders inside folders.

**Fig : Tree- Structured Directory.**

---

### Advantages:

- User can access other user's files by specifying the path name.
- User can create his own sub-directories.
- Searching becomes very easy; we can use both absolute path as well as relative.

---

### Disadvantages:

- Every file does not fit into the hierarchical model; files may be saved into multiple directories.

College Sahayak

- We cannot share files.
- It is inefficient, because accessing a file may go under multiple directories.

315319

12526

### 3 Hours / 70 Marks

Instructions – (1) All Questions are Compulsory.

- Answer each next main Question on a new page.
- Illustrate your answers with neat sketches wherever necessary.
- Figures to the right indicate full marks.
- Assume suitable data, if necessary.
- Mobile Phone, Pager and any other Electronic Communication devices are not permissible in Examination Hall.

Marks

**Attempt any FIVE of the following : 10**

- State any four operating system services.
- Enlist four process control system calls.
- Define –
- Turnaround time
- Throughput
- What are major differences between Linux and windows ?
- Define thread. Enlist two benefits of thread.
- List any four file operations. (Any two)
- Define term page fault.

Attempt any THREE of the following : 12

- Explain real time system. List any two applications.
- Explain the concept of variable partitioning.
- Explain free space management techniques –
- Bitmap
- Linked List
- Describe two level directory structure, with advantages and disadvantages.

Attempt any THREE of the following : 12

- Draw and explain process state diagram.
- State any three file types along with their usual extensions. What is the need of file extension ?
- Differentiate between linked and Indexed file allocation. (Any four points)
- Explain any two multithreading models with suitable diagram.

Attempt any THREE of the following : 12

- Describe First Fit, Best Fit, Worst Fit along with example.
- Describe necessary conditions leading to deadlock.

Differentiate between paging and segmentation. (Any four points)

- Explain process control block with the help of neat diagram.
- State the meaning of scheduler. Explain any one type of scheduler in detail.

Attempt any TWO of the following : 12

Consider the following processes.

Find out average waiting time for the following algorithms :

- SJF
- FCFS
- Priority scheduling

College Sahayak

(Low number ■ High Priority)

- What is the output of following Linux commands.
- sleep 12
- Ps - u User1
- wait 2535088
- Define terms deadlock, deadlock avoidance, deadlock prevention. Describe in brief the banker's algorithm for dead lock avoidance.

Attempt any TWO of the following : 12

- What is operating system ? Explain following types of operating system with suitable example of each.
- Distributed OS
- Mobile OS
- Describe Shortest Remaining Time Next (SRTN) and Round Robin Scheduling algorithms with suitable example.
- Consider the following page reference string arrival with three page frames 5, 6, 7, 8, 9, 7, 8, 5, 9, 7, 8, 7, 9, 6, 5, 06.

Calculate number of page faults with FIFO (First In First Out) and optimal page replacement algorithms.

College Sahayak

# BOARD EXAM — SOLVED PYQ EXAMPLES

*The following questions are sourced from MSBTE board examination papers (315319). Practice these for Summer / Winter examination preparation.*

**Q1. Course Code 315319**

**Q2. RATIONALE**

**Q3. INDUSTRY / EMPLOYER EXPECTED OUTCOME**

Interpret features of Operating System.

**Q4. COURSE LEVEL LEARNING OUTCOMES (COS)**

Students will be able to achieve & demonstrate the following COs on completion of course based learning

CO1 - Explain the services and components of an Operating System.

CO2 - Describe the different aspects of Process Management in an Operating System. CO3 - Implement various CPU Scheduling algorithms and evaluate their effectiveness. CO4 - Analyze the Memory Management techniques used by an Operating System.

CO5 - Apply techniques for effective File Management in an Operating System.

**Q5. TEACHING-LEARNING & ASSESSMENT SCHEME**

**Q6. THEORY LEARNING OUTCOMES AND ALIGNED COURSE CONTENT**

**Q7. LABORATORY LEARNING OUTCOME AND ALIGNED PRACTICAL / TUTORIAL EXPERIENCES.**

**Q8. SUGGESTED MICRO PROJECT / ASSIGNMENT/ ACTIVITIES FOR SPECIFIC LEARNING / SKILLS DEVELOPMENT (SELF LEARNING)**

**Q9. Assignment**

Find out the total number of page faults using – i) First In First Out ii) Least recently used page replacement ii) Optimal page replacement Page replacement algorithms of memory management, if the page are coming in the order 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

Compare between CLI based Operating System and GUI based Operating System.

Differentiate between process and thread (any two points). Also discuss the benefits of multithreaded programming.

Enlist different file allocation methods? Explain contiguous and indexed allocation method in detail.

**Q10. Micro project**

Create a report depicting features of different types of operating systems- Batch operating system, Multi programmed, Time shared, Multiprocessor systems, Real time systems, Mobile OS with examples.

Implement and Compare Memory Allocation Strategies - First Fit, Best Fit, Worst Fit

Create a report on different operating system tools used to perform various functions.

**Q11. Self learning**

Complete any one course related to the operating system on MOOCS such as NPTEL, Coursera, Infosys Springboard etc.

**Q12. LABORATORY EQUIPMENT / INSTRUMENTS / TOOLS / SOFTWARE REQUIRED**

**Q13. SUGGESTED WEIGHTAGE TO LEARNING EFFORTS & ASSESSMENT PURPOSE (Specification**

**Q14. Table)**

**Q15. ASSESSMENT METHODOLOGIES/TOOLS**

**Q16. Formative assessment (Assessment for Learning)**

Continuous assessment based on process and product related performance indicators. Each practical will be assessed considering 1) 60% weightage is to process 2) 40% weightage to product

**Q17. Summative Assessment (Assessment of Learning)**
End Semester Examination, Lab Performance, Viva-voce

**Q18. SUGGESTED COS - POS MATRIX FORM**

**Q19. SUGGESTED LEARNING MATERIALS / BOOKS**

**Q20. LEARNING WEBSITES & PORTALS**

**Q21. MSBTE Approval Dt. 24/02/2025 Semester - 5, K Scheme**
315319

25226

---

## 3 Hours / 70 Marks

Seat No.

Instructions – (1) All Questions are Compulsory.

Answer each next main Question on a new page.

Illustrate your answer with neat sketches wherever necessary.

Figures to the right indicate full marks.

Assume suitable data, if necessary.

Mobile Phone, Pager and any other Electronic Communication devices are not permissible in Examination Hall.

---

## Marks

---

## Attempt any FIVE of the following: 10

List the different states of process.

State the use of PS command and kill command in Linux.

Define non-preemptive scheduling.

State the function of CPU scheduler.

Define fragmentation with its types.

Define Virtual memory.

Describe direct access method with one example.

P.T.O.

Describe various activities performed by following operating system components:–

Main memory management.

Process management.

Describe many-to-one multithreading model with suitable diagram.

Explain any four scheduling criteria.

Describe Bitmap-free space management technique with example.

List disadvantages of contiguous file allocation method.

---

## Attempt any TWO of the following: 12

Describe any six services of operating system with example.

Define deadlock prevention. Explain any two methods of deadlock prevention.

College Sahayak

Explain working of Non-contiguous memory management technique-paging with diagram of paging hardware.

## Attempt any TWO of the following: 12

Explain Inter Process Communication (IPC). Describe shared memory in detail with their advantages and disadvantages.

Consider the reference string:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

with frame size 3. Calculate page fault using FIFO and optimal page replacement algorithm.

What is the average turnaround time for the following process using:

FCFS scheduling algorithm.

SJF non-preemptive scheduling algorithm.

College Sahayak